Portal Administrador

Backend de gestión de inventario con arquitectura hexagonal, seguridad JWT y trazabilidad de calidad

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento de Ciencias de la Computación
Aseguramiento de la Calidad de Software

Informe Técnico Don Gato Inventory

Commit documentado: `691fa0d`
`feat: initialize inventory management backend project with hexagonal architecture and security configuration`

66 archivos | **17 endpoints** | **JWT + SQA**

Equipo de trabajo

Mesias Mariscal
Gabriel Murillo
Denise Rea
Julio Viche

Mayo 2026

Índice general

Índice de tablas

Índice de figuras

Capítulo 1

Resumen Ejecutivo

Qué se construyó

Se inicializó un backend empresarial para la cafetería Don Gato, orientado a gestión de inventario, productos, categorías y movimientos de stock. La implementación usa Spring Boot, JPA, validaciones Jakarta, seguridad con JWT, arquitectura hexagonal y un bloque de SQA basado en el modelo de calidad de McCall.

El commit transforma el proyecto base en una API REST completa. El núcleo del sistema ya no depende de detalles de infraestructura: el dominio contiene las reglas del negocio, la capa de aplicación coordina casos de uso, los puertos expresan contratos, la infraestructura implementa persistencia y seguridad, y la capa REST expone endpoints validados para consumidores externos.

El resultado tiene una intención clara de calidad: proteger las operaciones de escritura con autenticación, impedir inventario negativo, dejar trazabilidad de cada ajuste de stock, centralizar errores y probar reglas críticas con JUnit. La arquitectura separa responsabilidades y facilita cambiar PostgreSQL/JPA, controladores o seguridad sin tocar el corazón del negocio.

1.1 Datos del Commit

Tabla 1.1: Resumen cuantitativo del commit documentado

Hash	`691fa0d`
Mensaje	`feat: initialize inventory management backend project with hexagonal architecture and security configuration`
Archivos cambiados	66
Inserciones	3129
Eliminaciones	64
Capas creadas	Dominio, aplicación, infraestructura, interfaces REST, SQA y pruebas
Endpoints REST	17 rutas documentadas

1.2 Objetivo Funcional

El sistema permite administrar:

- Categorías de productos de la cafetería.

- Productos con precio, stock, estado y categoría.
- Movimientos de inventario de tipo `ENTRADA` o `SALIDA`.
- Login con JWT para proteger operaciones que modifican datos.
- Respuestas de error consistentes para validación, negocio y autenticación.

Capítulo 2

Arquitectura Hexagonal

2.1 Idea Central

La arquitectura hexagonal, también conocida como puertos y adaptadores, protege el núcleo del negocio. En este proyecto, el dominio no conoce Spring, JPA, HTTP ni JWT. Las dependencias apuntan hacia adentro: los controladores llaman casos de uso, los casos de uso dependen de puertos, y la infraestructura implementa esos puertos.

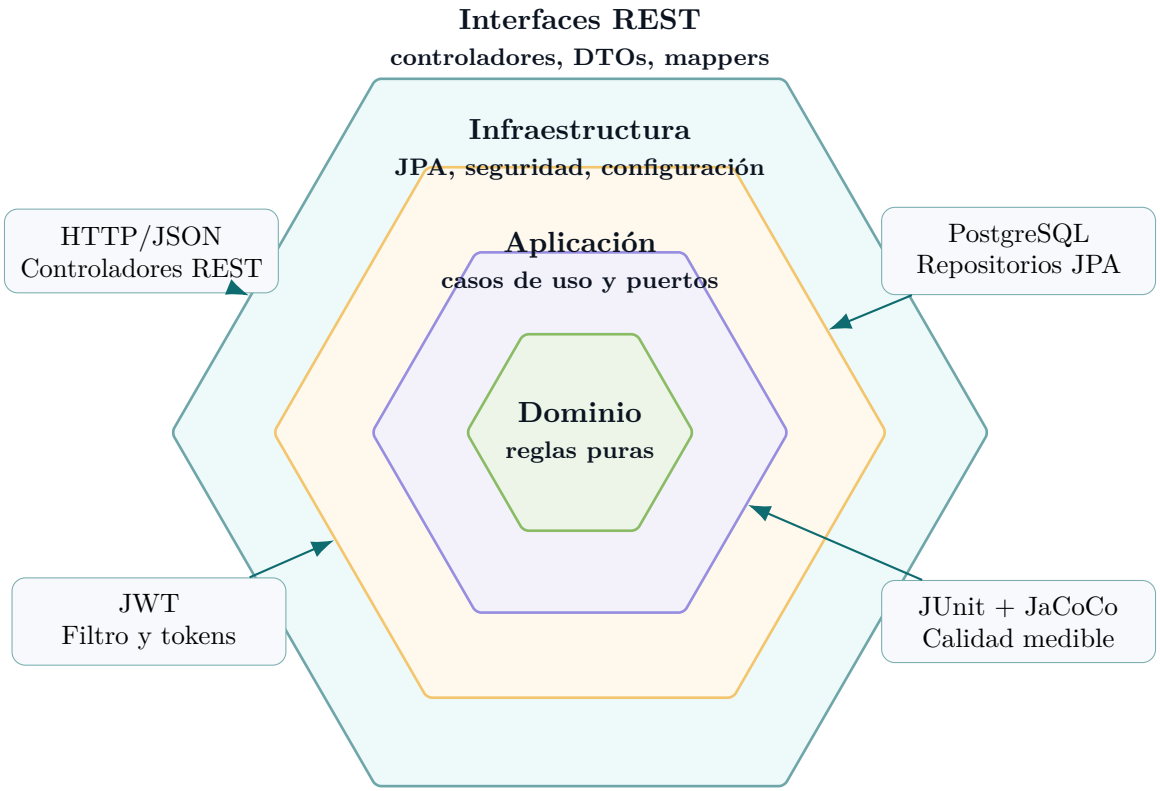

Figura 2.1: Lectura visual de la arquitectura hexagonal aplicada al backend.

2.2 Capas y Responsabilidades

Tabla 2.1: Responsabilidades principales por capa

Capa	Paquetes	Qué resuelve
Dominio	`domain.model`, `domain.exception`	Modela categorías, productos, movimientos, estados y reglas como stock no negativo, cantidades positivas y errores de negocio.
Aplicación	`application.usecases`, `application.ports.out`	Coordina transacciones, valida existencia de entidades, aplica reglas de inventario y habla con repositorios mediante puertos.
Infraestructura	`infrastructure.persistence`, `infrastructure.security`, `infrastructure.config`	Implementa persistencia JPA, entidades, mappers, seguridad JWT, CORS, usuarios en memoria y datos semilla.
Interfaces REST	`interfaces.rest.controller`, `dto`, `mapper`, `exception`	Expone la API HTTP, valida entradas, traduce DTOs y devuelve errores consistentes.
SQA	`sqa`, `src/test`, `.github/workflows`	Agrega motor de puntaje McCall, pruebas unitarias, JaCoCo y pipeline CI.

2.3 Flujo de Dependencias

Regla de dependencia

Las capas externas pueden conocer capas internas, pero las internas no deben conocer los detalles externos. Por eso `ProductUseCase` depende de `ProductRepositoryPort`, no de `JpaProductRepository`.

Ejemplo: caso de uso depende de puertos, no de JPA

```java
@Service
@RequiredArgsConstructor
@Slf4j
@Transactional(readOnly = true)
public class ProductUseCase {

    private final ProductRepositoryPort productRepository;
    private final CategoryRepositoryPort categoryRepository;

    @Transactional
    public Product create(Product product) {
        product.validate();
        validateCategoryExists(product.getCategoryId());
        product.recalculateStatus();
        product.setCreatedAt(LocalDateTime.now());
        return productRepository.save(product);
    }
}
```

Capítulo 3

Complejidad Técnica y Decisiones

3.1 Dominio Rico

El dominio no se limita a contener datos. En `Product` existen operaciones de negocio: recalcular estado, agregar stock, quitar stock y validar invariantes. Esta decisión reduce errores porque las reglas viajan con el objeto central del inventario.

Reglas críticas dentro del modelo Product

```
public void removeStock(int quantity) {
    if (quantity <= 0) {
        throw new IllegalArgumentException("Exit quantity must be positive");
    }
    int currentStock = (this.stock == null) ? 0 : this.stock;
    if (currentStock < quantity) {
        throw new IllegalStateException(
                String.format("Insufficient stock: available=%d, requested=%d", currentStock, quantity));
    }
    this.stock = currentStock - quantity;
    recalculateStatus();
}
```

3.2 Inventario Auditable

La actualización de stock no se hace directamente desde el endpoint de productos. El commit fuerza que el cambio de inventario pase por `StockMovementUseCase`. Así cada entrada o salida queda registrada con producto, cantidad, tipo, razón y fecha.

Decisión de integridad

El stock es una consecuencia de movimientos auditables. Esta es una decisión más segura que permitir edición libre de stock en `PUT /products/{id}`.

3.3 Complejidad Operacional

Tabla 3.1: Complejidad y costo conceptual de operaciones

Operación	Complejidad lógica	Comentario
Validar producto/categoría/movimiento	$O(1)$	Revisa campos simples y reglas locales.
Crear producto	$O(1)$ + I/O DB	Valida categoría, calcula estado, persiste. El costo real depende de latencia de base de datos.
Actualizar producto	$O(1)$ + I/O DB	Busca por ID, valida campos, no permite cambiar stock directo.
Registrar movimiento	$O(1)$ + I/O DB	Busca producto, aplica entrada/salida, guarda producto y movimiento dentro de transacción.
Listar productos/categorías/movimientos	$O(n)$	Retorna todos los registros; para producción convendría paginación.
Buscar productos por nombre	$O(n)$ o mejor con índice	La complejidad final depende del plan SQL y de índices disponibles.
Validar token JWT	$O(1)$ respecto al tamaño del sistema	Verifica firma HMAC y extrae usuario; luego carga detalles del usuario en memoria.

3.4 Transacciones

Los casos de uso están anotados con `@Transactional`. Las lecturas usan `readOnly=true`; las escrituras abren transacciones completas. En movimientos de stock, producto y movimiento se guardan como una unidad lógica: si falla la salida por stock insuficiente, no se debe persistir un movimiento incorrecto.

3.5 Modelo de Calidad de McCall

Tabla 3.2: Trazabilidad entre decisiones y factores McCall

Factor	Implementación	Evidencia
Correctness	Validaciones de dominio y DTOs	`Product.validate`, `StockMovement.validate`, Jakarta Validation
Integrity	JWT, stock auditable, errores sin stack-trace	`SecurityConfig`, `JwtAuthenticationFilter`, `GlobalExceptionHandler`
Testability	Pruebas aisladas de dominio y casos de uso	`ProductTest`, `StockMovementUseCaseTest`, JaCoCo
Maintainability	Capas separadas y mappers explícitos	REST mapper, persistence mapper, puertos de repositorio
Portability	Docker Compose y JPA	PostgreSQL containerizado, entidades ORM
Interoperability	API REST JSON	Controladores, DTOs y códigos HTTP claros

Capítulo 4

API REST y Endpoints

4.1 Reglas Generales

Base URL

Servidor local por defecto: `http://localhost:8080`. Todas las rutas públicas y protegidas están bajo `/api/v1`.

La seguridad configurada permite acceso público a `/api/v1/auth/**`, a lecturas de categorías y a lecturas de productos. El resto de operaciones requiere header:

Header requerido para operaciones protegidas

```
Authorization: Bearer <token-jwt>
```

4.2 Tabla Completa de Endpoints

Tabla 4.1: Todos los endpoints disponibles en el backend

Método	Endpoint	Seguridad	Uso
POST	`/api/v1/auth/login`	Público	Autentica usuario y devuelve JWT. Body: `AuthRequestDTO`. Respuesta: `AuthResponseDTO`.
GET	`/api/v1/categories`	Público	Lista todas las categorías. Respuesta: lista de `CategoryResponseDTO`.
GET	`/api/v1/categories/{id}`	Público	Obtiene categoría por ID. Devuelve 404 si no existe.
POST	`/api/v1/categories`	JWT requerido	Crea categoría. Body: `CategoryCreateDTO`. Respuesta 201.
PUT	`/api/v1/categories/{id}`	JWT requerido	Actualiza categoría completa. Body: `CategoryCreateDTO`.
DELETE	`/api/v1/categories/{id}`	JWT requerido	Elimina categoría por ID. Respuesta 204.

Método	Endpoint	Seguridad	Uso
GET	`/api/v1/products`	Público	Lista todos los productos. Respuesta: lista de `ProductResponseDTO`.
GET	`/api/v1/products/{id}`	Público	Obtiene producto por ID. Devuelve 404 si no existe.
GET	`/api/v1/products/category/{categoryId}`	Público	Lista productos asociados a una categoría existente.
GET	`/api/v1/products/status/{status}`	Público	Filtra por estado. Valores: `AVAILABLE`, `OUT_OF_STOCK`.
GET	`/api/v1/products/search?name={texto}`	Público	Busca productos por nombre ignorando mayúsculas/minúsculas.
POST	`/api/v1/products`	JWT requerido	Crea producto. Body: `ProductCreateDTO`. Respuesta 201.
PUT	`/api/v1/products/{id}`	JWT requerido	Actualiza nombre, precio y categoría. No actualiza stock directo.
DELETE	`/api/v1/products/{id}`	JWT requerido	Elimina producto por ID. Respuesta 204.
POST	`/api/v1/stock-movements`	JWT requerido	Registra entrada/salida y actualiza stock. Body: `StockMovementCreateDTO`. Respuesta 201.
GET	`/api/v1/stock-movements`	JWT requerido	Lista todos los movimientos de stock.
GET	`/api/v1/stock-movements/product/{productId}`	JWT requerido	Lista movimientos de un producto. Devuelve 404 si el producto no existe.

4.3 Contratos de Entrada

Tabla 4.2: DTOs de entrada y validaciones

DTO	Campos	Validaciones principales
`AuthRequestDTO`	`username`, `password`	Ambos obligatorios con `@NotBlank`.
`CategoryCreateDTO`	`name`, `description`	`name` obligatorio y máximo 100 caracteres; `description` máximo 255.
`ProductCreateDTO`	`name`, `price`, `stock`, `categoryId`	Nombre obligatorio, precio no negativo, stock inicial no negativo, categoría obligatoria.
`ProductUpdateDTO`	`name`, `price`, `categoryId`	Nombre y precio obligatorios; categoría opcional pero si llega debe existir.
`StockMovementCreateDTO`	`productId`, `type`, `quantity`, `reason`	Producto, tipo y razón obligatorios; cantidad mínima 1.

4.4 Enums Expuestos

Tabla 4.3: Valores enumerados permitidos

Enum	Valores
`ProductStatus`	`AVAILABLE`, `OUT_OF_STOCK`
`MovementType`	`ENTRADA`, `SALIDA`
`MovementReason`	`VENTA`, `REPOSICION`, `AJUSTE`, `DEVOLUCION`, `MERMA`

4.5 Ejemplos de Uso

Login

```
curl -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d "{\"username\":\"admin\",\"password\":\"dongato2026\"}"
```

Crear producto

```
curl -X POST http://localhost:8080/api/v1/products \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d "{\"name\":\"Latte\",\"price\":3.50,\"stock\":20,\"categoryId\":1}"
```

Registrar salida de stock

```
curl -X POST http://localhost:8080/api/v1/stock-movements \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d "{\"productId\":1,\"type\":\"SALIDA\",\"quantity\":2,\"reason\":\"VENTA\"}"
```

Capítulo 5

Seguridad

5.1 Autenticación JWT

El backend usa autenticación stateless. El usuario llama `/api/v1/auth/login`, Spring Security valida credenciales en memoria y `JwtTokenProvider` genera un token firmado con HMAC-SHA256. En cada request posterior, `JwtAuthenticationFilter` extrae el header `Authorization`, valida el token y registra la autenticación en el `SecurityContext`.

Reglas de autorización

```
.authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/v1/auth/**").permitAll()
        .requestMatchers(HttpMethod.GET, "/api/v1/categories/**").permitAll()
        .requestMatchers(HttpMethod.GET, "/api/v1/products/**").permitAll()
        .anyRequest().authenticated()
)
```

5.2 Usuarios de Prueba

Tabla 5.1: Usuarios configurados en memoria

Usuario	Contraseña	Rol
`admin`	`dongato2026`	`ADMIN`
`cajero`	`cafe123`	`USER`

Nota de endurecimiento

Para producción, estos usuarios deben migrarse a una tabla de usuarios, la clave JWT debe vivir en variables de entorno o gestor de secretos, y se debería agregar refresh token, rotación de claves, políticas de expiración y auditoría de intentos fallidos.

5.3 CORS y Sesiones

La API está preparada para frontends locales en `http://localhost:3000` y `http://localhost:5173`. La política usa sesiones stateless y desactiva CSRF porque la API no depende de cookies de sesión.

Capítulo 6

Persistencia y Datos

6.1 Modelo Persistente

La infraestructura contiene entidades JPA separadas de los modelos de dominio. Esta duplicación es intencional: el dominio representa reglas y el modelo persistente representa tablas. Los mappers traducen entre ambos mundos.

Por qué usar mappers

Los mappers evitan que anotaciones ORM y detalles de base de datos contaminen los modelos de negocio. Esto mejora mantenibilidad y portabilidad.

6.2 Repositorios

Los repositorios JPA implementan consultas derivadas como:

- Buscar productos por categoría.
- Buscar productos por estado.
- Buscar por nombre ignorando mayúsculas/minúsculas.
- Listar movimientos por producto.

6.3 Docker Compose

El servicio PostgreSQL queda configurado con imagen `postgres:16-alpine`, base `dongato_db`, usuario `dongato_user`, volumen persistente y healthcheck con `pg_isready`.

Capítulo 7

Manejo de Errores

7.1 Formato Común

Todos los errores se empaquetan en `ApiErrorDTO`, con estado HTTP, nombre del error, mensaje, ruta, timestamp y errores de campo cuando aplica.

Respuesta de error estandarizada

```
ApiErrorDTO error = ApiErrorDTO.builder()
        .status(status.value())
        .error(status.getReasonPhrase())
        .message(message)
        .path(request.getRequestURI())
        .timestamp(LocalDateTime.now())
        .build();
```

7.2 Mapa de Excepciones

Tabla 7.1: Errores manejados por `GlobalExceptionHandler`

Excepción	HTTP	Motivo
`ResourceNotFoundException`	404	Recurso inexistente.
`BusinessRuleException`	409	Violación de regla de negocio, como stock insuficiente.
`IllegalArgumentException`	400	Argumento inválido en dominio o aplicación.
`MethodArgumentNotValidException`	400	Validación Jakarta fallida en DTO.
`BadCredentialsException`	401	Usuario o contraseña inválidos.
`Exception`	500	Error inesperado sin filtrar detalles internos.

Capítulo 8

Pruebas, SQA y CI/CD

8.1 Pruebas Implementadas

El commit agrega pruebas unitarias para dominio, casos de uso y motor de calidad. Las reglas más sensibles quedan verificadas sin depender de HTTP ni de base de datos real.

Tabla 8.1: Cobertura conceptual de pruebas

Archivo de prueba	Qué verifica
`ProductTest`	Validación de producto, entradas, salidas, stock insuficiente y cálculo automático de estado.
`CategoryTest`	Invariantes básicas de categoría.
`StockMovementTest`	Datos obligatorios y coherencia del movimiento.
`CategoryUseCaseTest`	Creación, consulta, actualización y eliminación con repositorio simulado.
`StockMovementUseCaseTest`	Entradas/salidas, persistencia de movimiento y error por stock insuficiente.
`QualityScoringEngineTest`	Fórmula de puntaje, penalizaciones y rangos de calificación.
`DonGatoInventoryApplicationTests`	Carga del contexto Spring.

8.2 Motor de Calidad McCall

El motor calcula un puntaje de 0 a 100 con ponderaciones:

$$\text{score} = 0{,}30(\text{testability}) + 0{,}30(\text{correctness}) + 0{,}20(\text{maintainability}) + 0{,}20(\text{integrity})$$

8.3 Pipeline

El workflow `ci-cd.yml` ejecuta checkout, prepara JDK, corre `mvn clean verify`, genera JaCoCo y sube reportes de cobertura y pruebas como artefactos.

Tabla 8.2: Ponderaciones del motor de calidad

Factor	Peso	Métrica usada
Correctness	30 %	Bugs detectados.
Testability	30 %	Porcentaje de cobertura.
Maintainability	20 %	Code smells.
Integrity	20 %	Vulnerabilidades; tolerancia cero.

Observación técnica

El `pom.xml` declara `java.version=25`, mientras el workflow configura JDK 21. Conviene alinear ambas versiones antes de una entrega formal para evitar diferencias entre entorno local y CI.

Capítulo 9

Inventario de Archivos del Commit

9.1 Resumen por Tipo de Cambio

Tabla 9.1: Distribución de archivos cambiados

Tipo	Cantidad	Detalle
Agregados	59	Nuevo backend por capas, DTOs, seguridad, SQA y tests.
Modificados	5	Pipeline, `.gitignore`, Docker Compose, `pom.xml`, propiedades.
Eliminados	2	Clases base del paquete anterior `com.sqa_inventory.backend`.
Total	66	Alcance completo del commit `691fa0d`.

9.2 Lista de Archivos

Tabla 9.2: Archivos activos documentados con código en el apéndice

Tipo	Ruta
M	`.github/workflows/ci-cd.yml`
M	`.gitignore`
M	`backend/compose.yaml`
M	`backend/pom.xml`
A	`backend/src/main/java/com/dongato/inventory/DonGatoInventoryApplication.java`
A	`backend/src/main/java/com/dongato/inventory/application/ports/out/CategoryRepositoryPort.java`
A	`backend/src/main/java/com/dongato/inventory/application/ports/out/ProductRepositoryPort.java`
A	`backend/src/main/java/com/dongato/inventory/application/ports/out/StockMovementRepositoryPort.java`
A	`backend/src/main/java/com/dongato/inventory/application/usecases/CategoryUseCase.java`
A	`backend/src/main/java/com/dongato/inventory/application/usecases/ProductUseCase.java`

Tipo	Ruta
A	backend/src/main/java/com/dongato/inventory/application/usecases/StockMovementUseCase.java
A	backend/src/main/java/com/dongato/inventory/domain/exception/BusinessRuleException.java
A	backend/src/main/java/com/dongato/inventory/domain/exception/InsufficientStockException.java
A	backend/src/main/java/com/dongato/inventory/domain/exception/ResourceNotFoundException.java
A	backend/src/main/java/com/dongato/inventory/domain/model/Category.java
A	backend/src/main/java/com/dongato/inventory/domain/model/MovementReason.java
A	backend/src/main/java/com/dongato/inventory/domain/model/MovementType.java
A	backend/src/main/java/com/dongato/inventory/domain/model/Product.java
A	backend/src/main/java/com/dongato/inventory/domain/model/ProductStatus.java
A	backend/src/main/java/com/dongato/inventory/domain/model/StockMovement.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/config/DataSeeder.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/adapter/CategoryRepositoryAdapter.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/adapter/ProductRepositoryAdapter.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/adapter/StockMovementRepositoryAdapter.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/entity/CategoryEntity.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/entity/ProductEntity.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/entity/StockMovementEntity.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/mapper/CategoryPersistenceMapper.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/mapper/ProductPersistenceMapper.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/mapper/StockMovementPersistenceMapper.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/repository/JpaCategoryRepository.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/repository/JpaProductRepository.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/repository/JpaStockMovementRepository.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/security/JwtAuthenticationFilter.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/security/JwtTokenProvider.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/security/SecurityConfig.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/AuthController.java

Tipo	Ruta
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/CategoryController.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/ProductController.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/StockMovementController.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ApiErrorDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/AuthRequestDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/AuthResponseDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/CategoryCreateDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/CategoryResponseDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ProductCreateDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ProductResponseDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ProductUpdateDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/StockMovementCreateDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/StockMovementResponseDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/exception/GlobalExceptionHandler.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/mapper/CategoryRestMapper.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/mapper/ProductRestMapper.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/mapper/StockMovementRestMapper.java
A	backend/src/main/java/com/dongato/inventory/sqa/QualityScoringEngine.java
M	backend/src/main/resources/application.properties
A	backend/src/test/java/com/dongato/inventory/DonGatoInventoryApplicationTests.java
A	backend/src/test/java/com/dongato/inventory/application/usecases/CategoryUseCaseTest.java
A	backend/src/test/java/com/dongato/inventory/application/usecases/StockMovementUseCaseTest.java
A	backend/src/test/java/com/dongato/inventory/domain/model/CategoryTest.java
A	backend/src/test/java/com/dongato/inventory/domain/model/ProductTest.java
A	backend/src/test/java/com/dongato/inventory/domain/model/StockMovementTest.java
A	backend/src/test/java/com/dongato/inventory/sqa/QualityScoringEngineTest.java
A	backend/src/test/resources/application-test.properties

Los dos archivos eliminados se incluyen como snapshot textual al final del apéndice para conservar la trazabilidad del total de 66 cambios.

Apéndice A

Código Completo de los Archivos Activos

A.1 Configuración de Proyecto

.github/workflows/ci-cd.yml

Código fuente

```yaml
name: Don Gato Inventory - DevSecOps Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    name: Build, Test & Security Scan
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: ./backend

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: 'maven'

      - name: Build and Test with Maven
        run: mvn clean verify -DskipDocker=true

      - name: Generate JaCoCo Coverage Report
        run: mvn jacoco:report

      - name: Upload Coverage Report
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: backend/target/site/jacoco/

      - name: Upload Test Results
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: test-results
          path: backend/target/surefire-reports/
```

.gitignore

Código fuente

```
# Compiled class file
*.class

# Log file
*.log

# BlueJ files
*.ctxt

# Mobile Tools for Java (J2ME)
.mtj.tmp/

# Package Files
*.jar
*.war
*.nar
*.ear
*.zip
*.tar.gz
*.rar

# virtual machine crash logs
hs_err_pid*
replay_pid*

# Maven
backend/target/

# IDE
.idea/
*.iml
.vscode/

# OS
.DS_Store
Thumbs.db

# Environment
.env
*.env.local

# LaTeX build artifacts (keep .tex and .pdf, ignore the rest)
*.aux
*.bbl
*.blg
*.fdb_latexmk
*.fls
*.listing
*.lof
*.log
*.lot
*.out
*.synctex.gz
*.toc
*.xdv
```

backend/compose.yaml

Código fuente

```
services:
  postgres:
    image: 'postgres:16-alpine'
    container_name: dongato-postgres
    environment:
      POSTGRES_DB: dongato_db
```

```
      POSTGRES_USER: dongato_user
      POSTGRES_PASSWORD: dongato123
    ports:
      - '5432:5432'
    volumes:
      - dongato_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U dongato_user -d dongato_db"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped

volumes:
  dongato_data:
    driver: local
```

backend/pom.xml

Código fuente

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.0.6</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.dongato</groupId>
  <artifactId>inventory</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <name>dongato-inventory</name>
  <description>Cafeteria Don Gato - Inventory Management System (SQA Project)</description>

  <properties>
    <java.version>25</java.version>
    <jjwt.version>0.12.6</jjwt.version>
  </properties>

  <dependencies>
    <!-- Web & REST (Interoperability) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>

    <!-- Persistence / JPA (Integrity, Reliability) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- Validation (Correctness) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <!-- Security (Integrity) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <!-- JWT (Integrity - Authentication) -->
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-api</artifactId>
```

```
    <version>${jjwt.version}</version>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>${jjwt.version}</version>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>${jjwt.version}</version>
    <scope>runtime</scope>
  </dependency>

  <!-- Docker Compose integration (Portability) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-docker-compose</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>

  <!-- PostgreSQL driver -->
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>

  <!-- DevTools (Maintainability) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>

  <!-- Lombok (Maintainability) -->
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>

  <!-- ========== TEST DEPENDENCIES ========== -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <excludes>
          <exclude>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
          </exclude>
        </excludes>
```

```
      </configuration>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <executions>
        <execution>
          <id>default-compile</id>
          <phase>compile</phase>
          <goals>
            <goal>compile</goal>
          </goals>
          <configuration>
            <annotationProcessorPaths>
              <path>
                <groupId>org.projectlombok</groupId>
                <artifactId>lombok</artifactId>
              </path>
            </annotationProcessorPaths>
          </configuration>
        </execution>
        <execution>
          <id>default-testCompile</id>
          <phase>test-compile</phase>
          <goals>
            <goal>testCompile</goal>
          </goals>
          <configuration>
            <annotationProcessorPaths>
              <path>
                <groupId>org.projectlombok</groupId>
                <artifactId>lombok</artifactId>
              </path>
            </annotationProcessorPaths>
          </configuration>
        </execution>
      </executions>
    </plugin>
    <!-- JaCoCo for Code Coverage (Testability) -->
    <plugin>
      <groupId>org.jacoco</groupId>
      <artifactId>jacoco-maven-plugin</artifactId>
      <version>0.8.14</version>
      <executions>
        <execution>
          <goals>
            <goal>prepare-agent</goal>
          </goals>
        </execution>
        <execution>
          <id>report</id>
          <phase>test</phase>
          <goals>
            <goal>report</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>

</project>
```

backend/src/main/resources/application.properties

Código fuente

```
# ================================================================================
# Cafeteria Don Gato - Inventory Management System
# ================================================================================

```

```
spring.application.name=dongato-inventory

# --- Database (auto-configured by spring-boot-docker-compose) ---
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.open-in-view=false

# --- Server ---
server.port=8080
server.error.include-message=always
server.error.include-binding-errors=always

# --- Security Headers ---
server.servlet.session.cookie.http-only=true
server.servlet.session.cookie.secure=true

# --- Logging ---
logging.level.root=INFO
logging.level.com.dongato.inventory=DEBUG
logging.level.org.springframework.security=INFO
```

backend/src/test/resources/application-test.properties

Código fuente

```
# ============================================================================
# Test Profile Configuration
# ============================================================================

spring.application.name=dongato-inventory-test

# Use H2 in-memory database for tests
spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=false
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect
spring.jpa.show-sql=false
spring.jpa.open-in-view=false

# Disable Docker Compose in tests
spring.docker.compose.enabled=false

# JWT config for tests
jwt.secret=TestSecretKeyForJWTTokenMustBeAtLeast256BitsLongForHMACSHA
jwt.expiration-ms=3600000

logging.level.root=WARN
```

A.2 Aplicación

backend/src/main/java/com/dongato/inventory/DonGatoInventoryApplication.java

Código fuente

```
package com.dongato.inventory;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
```

```
 * Cafetería Don Gato - Inventory Management System.
 * <p>
 * Entry point for the Spring Boot application.
 * Uses Clean Architecture with McCall quality model traceability.
 */
@SpringBootApplication
public class DonGatoInventoryApplication {

    public static void main(String[] args) {
        SpringApplication run DonGatoInventoryApplication class, args);
    }
}
```

A.3 Dominio

backend/src/main/java/com/dongato/inventory/domain/model/Category.java

Código fuente

```
package com.dongato.inventory.domain.model;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/**
 * Domain model for Category.
 * <p>
 * Represents a product grouping (e.g., CAFE, BEBIDA, SNACK).
 * McCall Factor: Correctness -- pure business rules without framework coupling.
 */
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class Category {

    private Long id;
    private String name;
    private String description;
    private LocalDateTime createdAt;

    /**
     * Validates that the category has a non-blank name.
     *
     * @throws IllegalArgumentException if the name is blank
     */
    public void validate() {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Category name cannot be empty");
        }
    }
}
```

backend/src/main/java/com/dongato/inventory/domain/model/Product.java

Código fuente

```
package com.dongato.inventory.domain.model;

import lombok.AllArgsConstructor;
import lombok.Builder;
```

```java
import lombok.Data;
import lombok.NoArgsConstructor;

import java.math.BigDecimal;
import java.time.LocalDateTime;

/**
 * Domain model for Product -- the heart of the inventory.
 * <p>
 * Contains business logic for stock management and automatic status calculation.
 * McCall Factor: Correctness -- encapsulates invariants that prevent invalid states.
 */
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class Product {

    private Long id;
    private String name;
    private BigDecimal price;
    private Integer stock;
    private ProductStatus status;
    private Long categoryId;
    private LocalDateTime createdAt;

    /**
     * Automatically recalculates the product status based on current stock level.
     * Stock <= 0 results in OUT_OF_STOCK, otherwise AVAILABLE.
     */
    public void recalculateStatus() {
        this.status = (this.stock != null && this.stock > 0)
                ? ProductStatus.AVAILABLE
                : ProductStatus.OUT_OF_STOCK;
    }

    /**
     * Applies a stock entry (adds quantity).
     *
     * @param quantity the quantity to add (must be positive)
     * @throws IllegalArgumentException if quantity is not positive
     */
    public void addStock(int quantity) {
        if (quantity <= 0) {
            throw new IllegalArgumentException("Entry quantity must be positive");
        }
        this.stock = (this.stock == null ? 0 : this.stock) + quantity;
        recalculateStatus();
    }

    /**
     * Applies a stock exit (subtracts quantity).
     *
     * @param quantity the quantity to subtract (must be positive)
     * @throws IllegalArgumentException if quantity is not positive
     * @throws IllegalStateException    if insufficient stock
     */
    public void removeStock(int quantity) {
        if (quantity <= 0) {
            throw new IllegalArgumentException("Exit quantity must be positive");
        }
        int currentStock = (this.stock == null) ? 0 : this.stock;
        if (currentStock < quantity) {
            throw new IllegalStateException(
                    String.format("Insufficient stock: available=%d, requested=%d", currentStock, quantity)
    );
        }
        this.stock = currentStock - quantity;
        recalculateStatus();
    }

    /**
     * Validates basic product invariants.
     */
    public void validate() {
```

```
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Product name cannot be empty");
        }
        if (price == null || price.compareTo(BigDecimal.ZERO) < 0) {
            throw new IllegalArgumentException("Product price must be non-negative");
        }
        if (stock != null && stock < 0) {
            throw new IllegalArgumentException("Product stock cannot be negative");
        }
    }
}
```

backend/src/main/java/com/dongato/inventory/domain/model/ProductStatus.java

Código fuente

```
package com.dongato.inventory.domain.model;

/**
 * Represents the status of a product in the inventory.
 * <p>
 * McCall Factor: Correctness -- ensures valid product states.
 */
public enum ProductStatus {
    AVAILABLE,
    OUT_OF_STOCK
}
```

backend/src/main/java/com/dongato/inventory/domain/model/StockMovement.java

Código fuente

```
package com.dongato.inventory.domain.model;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/**
 * Domain model for StockMovement -- audit trail for inventory changes.
 * <p>
 * Records every stock change with type (ENTRADA/SALIDA) and reason.
 * McCall Factor: Integrity -- enables full traceability of inventory operations.
 */
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class StockMovement {

    private Long id;
    private Long productId;
    private MovementType type;
    private Integer quantity;
    private MovementReason reason;
    private LocalDateTime createdAt;

    /**
     * Validates that the movement has coherent data.
     */
    public void validate() {
        if (productId == null) {
            throw new IllegalArgumentException("Product ID is required for stock movement");
        }
```

```
        if (type == null) {
            throw new IllegalArgumentException("Movement type is required");
        }
        if (quantity == null || quantity <= 0) {
            throw new IllegalArgumentException("Movement quantity must be positive");
        }
        if (reason == null) {
            throw new IllegalArgumentException("Movement reason is required");
        }
    }
}
```

backend/src/main/java/com/dongato/inventory/domain/model/MovementType.java

Código fuente

```
package com.dongato.inventory.domain.model;

/**
 * Type of stock movement.
 * <p>
 * McCall Factor: Integrity -- ensures auditability of inventory changes.
 */
public enum MovementType {
    ENTRADA,
    SALIDA
}
```

backend/src/main/java/com/dongato/inventory/domain/model/MovementReason.java

Código fuente

```
package com.dongato.inventory.domain.model;

/**
 * Reason for a stock movement.
 * <p>
 * McCall Factor: Integrity -- provides context for audit trail.
 */
public enum MovementReason {
    VENTA,
    REPOSICION,
    AJUSTE,
    DEVOLUCION,
    MERMA
}
```

backend/src/main/java/com/dongato/inventory/domain/exception/BusinessRuleException.java

Código fuente

```
package com.dongato.inventory.domain.exception;

/**
 * Thrown when a business rule is violated.
 * McCall Factor: Correctness -- enforces business invariants at domain level.
 */
public class BusinessRuleException extends RuntimeException {

    public BusinessRuleException(String message) {
        super(message);
    }
```

```
}
```

backend/src/main/java/com/dongato/inventory/domain/exception/InsufficientStockException.java

Código fuente

```java
package com.dongato.inventory.domain.exception;

/**
 * Thrown when there is insufficient stock for an operation.
 * McCall Factor: Correctness -- prevents inventory going negative.
 */
public class InsufficientStockException extends BusinessRuleException {

    public InsufficientStockException(String productName, int available, int requested) {
        super(String.format("Insufficient stock for '%s': available=%d, requested=%d",
                productName, available, requested));
    }
}
```

backend/src/main/java/com/dongato/inventory/domain/exception/ResourceNotFoundException.java

Código fuente

```java
package com.dongato.inventory.domain.exception;

/**
 * Thrown when a requested resource is not found.
 * McCall Factor: Correctness -- clear error semantics.
 */
public class ResourceNotFoundException extends RuntimeException {

    public ResourceNotFoundException(String resourceName, Long id) {
        super(String.format("%s not found with id: %d", resourceName, id));
    }

    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

A.4 Aplicación: Puertos y Casos de Uso

backend/src/main/java/com/dongato/inventory/application/ports/out/CategoryRepositoryPort.java

Código fuente

```java
package com.dongato.inventory.application.ports.out;

import com.dongato.inventory.domain.model.Category;

import java.util.List;
import java.util.Optional;

/**
 * Output port for Category persistence.
 * <p>
 * McCall Factor: Reusability -- abstracts persistence from business logic (DIP).
 */
public interface CategoryRepositoryPort {
```

```
14
15      Category save(Category category);
16
17      Optional<Category> findById Long id);
18
19      List<Category> findAll();
20
21      boolean existsById(Long id);
22
23      boolean existsByName String name);
24
25      void deleteById Long id);
26  }
```

backend/src/main/java/com/dongato/inventory/application/ports/out/ProductRepositoryPort.java

Código fuente

```
1   package com.dongato.inventory.application.ports.out;
2
3   import com.dongato.inventory.domain.model.Product;
4   import com.dongato.inventory.domain.model.ProductStatus;
5
6   import java.util.List;
7   import java.util.Optional;
8
9   /**
10   * Output port for Product persistence.
11   * <p>
12   * McCall Factor: Reusability -- decouples domain from infrastructure.
13   */
14  public interface ProductRepositoryPort {
15
16      Product save(Product product);
17
18      Optional<Product> findById(Long id);
19
20      List<Product> findAll();
21
22      List<Product> findByCategoryId(Long categoryId);
23
24      List<Product> findByStatus(ProductStatus status);
25
26      List<Product> findByNameContainingIgnoreCase(String name);
27
28      boolean existsById(Long id);
29
30      void deleteById(Long id);
31  }
```

backend/src/main/java/com/dongato/inventory/application/ports/out/StockMovementRepositoryPort.java

Código fuente

```
1   package com.dongato.inventory.application.ports.out;
2
3   import com.dongato.inventory.domain.model.StockMovement;
4
5   import java.util.List;
6
7   /**
8    * Output port for StockMovement persistence.
9    * <p>
10   * McCall Factor: Integrity -- provides the contract for audit trail storage.
11   */
12  public interface StockMovementRepositoryPort {
13
```

```
    StockMovement save(StockMovement movement);

    List<StockMovement> findByProductId(Long productId);

    List<StockMovement> findAll();
}
```

backend/src/main/java/com/dongato/inventory/application/usecases/CategoryUseCase.java

Código fuente

```java
package com.dongato.inventory.application.usecases;

import com.dongato.inventory.application.ports.out.CategoryRepositoryPort;
import com.dongato.inventory.domain.exception.BusinessRuleException;
import com.dongato.inventory.domain.exception.ResourceNotFoundException;
import com.dongato.inventory.domain.model.Category;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.List;

/**
 * Use case for Category management.
 * <p>
 * McCall Factors: Correctness (business rules), Reusability (framework-agnostic logic).
 */
@Service
@RequiredArgsConstructor
@Slf4j
@Transactional(readOnly = true)
public class CategoryUseCase {

    private final CategoryRepositoryPort categoryRepository;

    @Transactional
    public Category create(Category category) {
        category.validate();

        if (categoryRepository.existsByName(category.getName())) {
            throw new BusinessRuleException(
                    String.format("Category with name '%s' already exists", category.getName()));
        }

        category.setCreatedAt(LocalDateTime.now());
        log.info("Creating category: {}", category.getName());
        return categoryRepository.save(category);
    }

    public Category findById(Long id) {
        return categoryRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Category", id));
    }

    public List<Category> findAll() {
        return categoryRepository.findAll();
    }

    @Transactional
    public Category update(Long id, Category updated) {
        Category existing = findById(id);
        updated.validate();

        existing.setName(updated.getName());
        existing.setDescription(updated.getDescription());

        log.info("Updating category id={}: {}", id, existing.getName());
        return categoryRepository.save(existing);
```

```
    }

    @Transactional
    public void delete(Long id) {
        if (!categoryRepository.existsById(id)) {
            throw new ResourceNotFoundException("Category", id);
        }
        log.info("Deleting category id={}", id);
        categoryRepository.deleteById(id);
    }
}
```

backend/src/main/java/com/dongato/inventory/application/usecases/ProductUseCase.java

Código fuente

```
package com.dongato.inventory.application.usecases;

import com.dongato.inventory.application.ports.out.CategoryRepositoryPort;
import com.dongato.inventory.application.ports.out.ProductRepositoryPort;
import com.dongato.inventory.domain.exception.ResourceNotFoundException;
import com.dongato.inventory.domain.model.Product;
import com.dongato.inventory.domain.model.ProductStatus;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.List;

/**
 * Use case for Product management.
 * <p>
 * McCall Factors: Correctness (validation), Reusability (port-based), Integrity (status tracking).
 */
@Service
@RequiredArgsConstructor
@Slf4j
@Transactional(readOnly = true)
public class ProductUseCase {

    private final ProductRepositoryPort productRepository;
    private final CategoryRepositoryPort categoryRepository;

    @Transactional
    public Product create(Product product) {
        product.validate();
        validateCategoryExists(product.getCategoryId());

        if (product.getStock() == null) {
            product.setStock(0);
        }
        product.recalculateStatus();
        product.setCreatedAt(LocalDateTime.now());

        log.info("Creating product: {} (category={})", product.getName(), product.getCategoryId());
        return productRepository.save(product);
    }

    public Product findById(Long id) {
        return productRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Product", id));
    }

    public List<Product> findAll() {
        return productRepository.findAll();
    }

    public List<Product> findByCategoryId(Long categoryId) {
        validateCategoryExists(categoryId);
```

```
        return productRepository.findByCategoryId(categoryId);
    }

    public List<Product> findByStatus(ProductStatus status) {
        return productRepository.findByStatus(status);
    }

    public List<Product> searchByName(String name) {
        return productRepository.findByNameContainingIgnoreCase(name);
    }

    @Transactional
    public Product update(Long id, Product updated) {
        Product existing = findById(id);
        updated.validate();

        if (updated.getCategoryId() != null) {
            validateCategoryExists(updated.getCategoryId());
            existing.setCategoryId(updated.getCategoryId());
        }

        existing.setName(updated.getName());
        existing.setPrice(updated.getPrice());
        // Stock is NOT updated directly here -- use StockMovement for traceability
        existing.recalculateStatus();

        log.info("Updating product id={}: {}", id, existing.getName());
        return productRepository.save(existing);
    }

    @Transactional
    public void delete(Long id) {
        if (!productRepository.existsById(id)) {
            throw new ResourceNotFoundException("Product", id);
        }
        log.info("Deleting product id={}", id);
        productRepository.deleteById(id);
    }

    private void validateCategoryExists(Long categoryId) {
        if (categoryId != null && !categoryRepository.existsById(categoryId)) {
            throw new ResourceNotFoundException("Category", categoryId);
        }
    }
}
```

backend/src/main/java/com/dongato/inventory/application/usecases/StockMovementUseCase.java

Código fuente

```
package com.dongato.inventory.application.usecases;

import com.dongato.inventory.application.ports.out.ProductRepositoryPort;
import com.dongato.inventory.application.ports.out.StockMovementRepositoryPort;
import com.dongato.inventory.domain.exception.InsufficientStockException;
import com.dongato.inventory.domain.exception.ResourceNotFoundException;
import com.dongato.inventory.domain.model.MovementType;
import com.dongato.inventory.domain.model.Product;
import com.dongato.inventory.domain.model.StockMovement;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.List;

/**
 * Use case for StockMovement management -- the audit trail for all inventory changes.
 * <p>
 * Every stock modification MUST go through this use case to ensure traceability.
```

```
 * McCall Factors: Integrity (audit trail), Correctness (stock consistency).
 */
@Service
@RequiredArgsConstructor
@Slf4j
@Transactional(readOnly = true)
public class StockMovementUseCase {

    private final StockMovementRepositoryPort movementRepository;
    private final ProductRepositoryPort productRepository;

    /**
     * Registers a stock movement and updates the product stock accordingly.
     * ENTRADA adds stock; SALIDA removes stock.
     *
     * @param movement the stock movement to register
     * @return the persisted stock movement
     */
    @Transactional
    public StockMovement registerMovement(StockMovement movement) {
        movement.validate();

        Product product = productRepository.findById(movement.getProductId())
                .orElseThrow(() -> new ResourceNotFoundException("Product", movement.getProductId()));

        if (movement.getType() == MovementType.ENTRADA) {
            product.addStock(movement.getQuantity());
            log.info("Stock ENTRY: product='{}', qty={}, reason={}",
                    product.getName(), movement.getQuantity(), movement.getReason());
        } else {
            try {
                product.removeStock(movement.getQuantity());
            } catch (IllegalStateException e) {
                throw new InsufficientStockException(
                        product.getName(), product.getStock(), movement.getQuantity());
            }
            log.info("Stock EXIT: product='{}', qty={}, reason={}",
                    product.getName(), movement.getQuantity(), movement.getReason());
        }

        productRepository.save(product);

        movement.setCreatedAt(LocalDateTime.now());
        return movementRepository.save(movement);
    }

    public List<StockMovement> findByProductId(Long productId) {
        if (!productRepository.existsById(productId)) {
            throw new ResourceNotFoundException("Product", productId);
        }
        return movementRepository.findByProductId(productId);
    }

    public List<StockMovement> findAll() {
        return movementRepository.findAll();
    }
}
```

A.5 Infraestructura: Persistencia

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/entity/CategoryEntity.java

Código fuente

```
package com.dongato.inventory.infrastructure.persistence.entity;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Builder;
```

```
import lombok Data;
import lombok NoArgsConstructor;

import java time LocalDateTime;

/**
 * JPA Entity for the 'category' table.
 * McCall Factor: Portability -- ORM abstracts database specifics.
 */
@Entity
@Table(name = "category")
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class CategoryEntity {

    @Id
    @GeneratedValue(strategy = GenerationType IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String name;

    @Column(length = 255)
    private String description;

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        if (this createdAt == null) {
            this createdAt = LocalDateTime now();
        }
    }
}
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/entity/ProductEntity.java

Código fuente

```
package com.dongato.inventory.infrastructure.persistence.entity;

import com.dongato.inventory.domain.model.ProductStatus;
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.math.BigDecimal;
import java.time.LocalDateTime;

/**
 * JPA Entity for the 'product' table.
 * McCall Factor: Portability -- ORM-managed persistence.
 */
@Entity
@Table(name = "product")
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class ProductEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "category_id", nullable = false)
```

```
    private Long categoryId;

    @Column(nullable = false, length = 150)
    private String name;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;

    @Column(nullable = false)
    private Integer stock;

    @Enumerated EnumType STRING)
    @Column(nullable = false, length = 20)
    private ProductStatus status;

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id", insertable = false, updatable = false)
    private CategoryEntity category;

    @PrePersist
    protected void onCreate() {
        if (this.createdAt == null) {
            this.createdAt = LocalDateTime.now();
        }
    }
}
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/entity/StockMovementEntity.java

Código fuente

```java
package com.dongato.inventory.infrastructure.persistence.entity;

import com.dongato.inventory.domain.model.MovementReason;
import com.dongato.inventory.domain.model.MovementType;
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/**
 * JPA Entity for the 'stock_movement' table.
 * McCall Factor: Integrity -- persists audit trail for inventory changes.
 */
@Entity
@Table(name = "stock_movement")
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class StockMovementEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "product_id", nullable = false)
    private Long productId;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 10)
    private MovementType type;

    @Column(nullable = false)
    private Integer quantity;
```

```
38
39      @Enumerated EnumType STRING)
40      @Column(nullable = false, length = 20)
41      private MovementReason reason;
42
43      @Column(name = "created_at", nullable = false, updatable = false)
44      private LocalDateTime createdAt;
45
46      @ManyToOne(fetch = FetchType.LAZY)
47      @JoinColumn(name = "product_id", insertable = false, updatable = false)
48      private ProductEntity product;
49
50      @PrePersist
51      protected void onCreate() {
52          if (this.createdAt == null) {
53              this.createdAt = LocalDateTime.now();
54          }
55      }
56  }
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/mapper/CategoryPersistenceMapper.java

Código fuente

```
1   package com.dongato.inventory.infrastructure.persistence.mapper;
2
3   import com.dongato.inventory.domain.model.Category;
4   import com.dongato.inventory.infrastructure.persistence.entity.CategoryEntity;
5
6   /**
7    * Maps between Category domain model and CategoryEntity JPA entity.
8    * <p>
9    * McCall Factor: Maintainability -- isolates domain from persistence representation.
10   */
11  public final class CategoryPersistenceMapper {
12
13      private CategoryPersistenceMapper() {
14          // Utility class -- prevent instantiation
15      }
16
17      public static Category toDomain(CategoryEntity entity) {
18          if (entity == null) return null;
19          return Category.builder()
20                  .id(entity.getId())
21                  .name(entity.getName())
22                  .description(entity.getDescription())
23                  .createdAt(entity.getCreatedAt())
24                  .build();
25      }
26
27      public static CategoryEntity toEntity(Category domain) {
28          if (domain == null) return null;
29          return CategoryEntity.builder()
30                  .id(domain.getId())
31                  .name(domain.getName())
32                  .description(domain.getDescription())
33                  .createdAt(domain.getCreatedAt())
34                  .build();
35      }
36  }
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/mapper/ProductPersistenceMapper.java

Código fuente

```java
package com.dongato.inventory.infrastructure.persistence.mapper;

import com.dongato.inventory.domain.model.Product;
import com.dongato.inventory.infrastructure.persistence.entity.ProductEntity;

/**
 * Maps between Product domain model and ProductEntity JPA entity.
 * <p>
 * McCall Factor: Maintainability -- decouples domain representation from ORM.
 */
public final class ProductPersistenceMapper {

    private ProductPersistenceMapper() {
        // Utility class -- prevent instantiation
    }

    public static Product toDomain(ProductEntity entity) {
        if (entity == null) return null;
        return Product.builder()
                .id(entity.getId())
                .name(entity.getName())
                .price(entity.getPrice())
                .stock(entity.getStock())
                .status(entity.getStatus())
                .categoryId(entity.getCategoryId())
                .createdAt(entity.getCreatedAt())
                .build();
    }

    public static ProductEntity toEntity(Product domain) {
        if (domain == null) return null;
        return ProductEntity.builder()
                .id(domain.getId())
                .name(domain.getName())
                .price(domain.getPrice())
                .stock(domain.getStock())
                .status(domain.getStatus())
                .categoryId(domain.getCategoryId())
                .createdAt(domain.getCreatedAt())
                .build();
    }
}
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/mapper/StockMovementPersistenceMapper.java

Código fuente

```java
package com.dongato.inventory.infrastructure.persistence.mapper;

import com.dongato.inventory.domain.model.StockMovement;
import com.dongato.inventory.infrastructure.persistence.entity.StockMovementEntity;

/**
 * Maps between StockMovement domain model and StockMovementEntity JPA entity.
 * <p>
 * McCall Factor: Maintainability -- isolates audit data from persistence details.
 */
public final class StockMovementPersistenceMapper {

    private StockMovementPersistenceMapper() {
        // Utility class -- prevent instantiation
    }

    public static StockMovement toDomain(StockMovementEntity entity) {
        if (entity == null) return null;
        return StockMovement.builder()
```

```
                .id(entity.getId())
                .productId(entity.getProductId())
                .type(entity.getType())
                .quantity(entity.getQuantity())
                .reason(entity.getReason())
                .createdAt(entity.getCreatedAt())
                .build();
    }

    public static StockMovementEntity toEntity(StockMovement domain) {
        if (domain == null) return null;
        return StockMovementEntity.builder()
                .id(domain.getId())
                .productId(domain.getProductId())
                .type(domain.getType())
                .quantity(domain.getQuantity())
                .reason(domain.getReason())
                .createdAt(domain.getCreatedAt())
                .build();
    }
}
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/repository/JpaCategoryRepository.java

Código fuente

```java
package com.dongato.inventory.infrastructure.persistence.repository;

import com.dongato.inventory.infrastructure.persistence.entity.CategoryEntity;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

/**
 * Spring Data JPA repository for Category.
 */
@Repository
public interface JpaCategoryRepository extends JpaRepository<CategoryEntity, Long> {

    boolean existsByNameIgnoreCase(String name);
}
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/repository/JpaProductRepository.java

Código fuente

```java
package com.dongato.inventory.infrastructure.persistence.repository;

import com.dongato.inventory.domain.model.ProductStatus;
import com.dongato.inventory.infrastructure.persistence.entity.ProductEntity;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

/**
 * Spring Data JPA repository for Product.
 */
@Repository
public interface JpaProductRepository extends JpaRepository<ProductEntity, Long> {

    List<ProductEntity> findByCategoryId(Long categoryId);

    List<ProductEntity> findByStatus(ProductStatus status);

    List<ProductEntity> findByNameContainingIgnoreCase(String name);
}
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/repository/JpaStockMovementRepository.java

Código fuente

```java
package com.dongato.inventory.infrastructure.persistence.repository;

import com.dongato.inventory.infrastructure.persistence.entity.StockMovementEntity;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

/**
 * Spring Data JPA repository for StockMovement.
 */
@Repository
public interface JpaStockMovementRepository extends JpaRepository<StockMovementEntity, Long> {

    List<StockMovementEntity> findByProductIdOrderByCreatedAtDesc(Long productId);
}
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/adapter/CategoryRepositoryAdapter.java

Código fuente

```java
package com.dongato.inventory.infrastructure.persistence.adapter;

import com.dongato.inventory.application.ports.out.CategoryRepositoryPort;
import com.dongato.inventory.domain.model.Category;
import com.dongato.inventory.infrastructure.persistence.mapper.CategoryPersistenceMapper;
import com.dongato.inventory.infrastructure.persistence.repository.JpaCategoryRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

/**
 * Adapter that implements CategoryRepositoryPort using Spring Data JPA.
 * <p>
 * McCall Factor: Portability -- swappable persistence implementation.
 */
@Component
@RequiredArgsConstructor
public class CategoryRepositoryAdapter implements CategoryRepositoryPort {

    private final JpaCategoryRepository jpaRepository;

    @Override
    public Category save(Category category) {
        var entity = CategoryPersistenceMapper.toEntity(category);
        var saved = jpaRepository.save(entity);
        return CategoryPersistenceMapper.toDomain(saved);
    }

    @Override
    public Optional<Category> findById(Long id) {
        return jpaRepository.findById(id)
                .map(CategoryPersistenceMapper::toDomain);
    }

    @Override
    public List<Category> findAll() {
        return jpaRepository.findAll().stream()
                .map(CategoryPersistenceMapper::toDomain)
                .collect(Collectors.toList());
    }

    @Override
```

```
    public boolean existsById(Long id) {
        return jpaRepository.existsById(id);
    }

    @Override
    public boolean existsByName(String name) {
        return jpaRepository.existsByNameIgnoreCase(name);
    }

    @Override
    public void deleteById(Long id) {
        jpaRepository.deleteById(id);
    }
}
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/adapter/ProductRepositoryAdapter.java

Código fuente

```
package com.dongato.inventory.infrastructure.persistence.adapter;

import com.dongato.inventory.application.ports.out.ProductRepositoryPort;
import com.dongato.inventory.domain.model.Product;
import com.dongato.inventory.domain.model.ProductStatus;
import com.dongato.inventory.infrastructure.persistence.mapper.ProductPersistenceMapper;
import com.dongato.inventory.infrastructure.persistence.repository.JpaProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

/**
 * Adapter that implements ProductRepositoryPort using Spring Data JPA.
 * <p>
 * McCall Factor: Portability -- database-agnostic through JPA abstraction.
 */
@Component
@RequiredArgsConstructor
public class ProductRepositoryAdapter implements ProductRepositoryPort {

    private final JpaProductRepository jpaRepository;

    @Override
    public Product save(Product product) {
        var entity = ProductPersistenceMapper.toEntity(product);
        var saved = jpaRepository.save(entity);
        return ProductPersistenceMapper.toDomain(saved);
    }

    @Override
    public Optional<Product> findById(Long id) {
        return jpaRepository.findById(id)
                .map(ProductPersistenceMapper::toDomain);
    }

    @Override
    public List<Product> findAll() {
        return jpaRepository.findAll().stream()
                .map(ProductPersistenceMapper::toDomain)
                .collect(Collectors.toList());
    }

    @Override
    public List<Product> findByCategoryId(Long categoryId) {
        return jpaRepository.findByCategoryId(categoryId).stream()
                .map(ProductPersistenceMapper::toDomain)
                .collect(Collectors.toList());
    }

```

```
    @Override
    public List Product> findByStatus(ProductStatus status) {
        return jpaRepository.findByStatus(status).stream()
                .map(ProductPersistenceMapper::toDomain)
                .collect(Collectors.toList());
    }

    @Override
    public List Product> findByNameContainingIgnoreCase(String name) {
        return jpaRepository.findByNameContainingIgnoreCase(name).stream()
                .map(ProductPersistenceMapper::toDomain)
                .collect(Collectors.toList());
    }

    @Override
    public boolean existsById(Long id) {
        return jpaRepository.existsById(id);
    }

    @Override
    public void deleteById(Long id) {
        jpaRepository.deleteById(id);
    }
}
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/adapter/StockMovementRepositoryAdapter.java

Código fuente

```java
package com.dongato.inventory.infrastructure.persistence.adapter;

import com.dongato.inventory.application.ports.out.StockMovementRepositoryPort;
import com.dongato.inventory.domain.model.StockMovement;
import com.dongato.inventory.infrastructure.persistence.mapper.StockMovementPersistenceMapper;
import com.dongato.inventory.infrastructure.persistence.repository.JpaStockMovementRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.stream.Collectors;

/**
 * Adapter that implements StockMovementRepositoryPort using Spring Data JPA.
 * <p>
 * McCall Factor: Integrity -- ensures audit data is persisted reliably.
 */
@Component
@RequiredArgsConstructor
public class StockMovementRepositoryAdapter implements StockMovementRepositoryPort {

    private final JpaStockMovementRepository jpaRepository;

    @Override
    public StockMovement save(StockMovement movement) {
        var entity = StockMovementPersistenceMapper.toEntity(movement);
        var saved = jpaRepository.save(entity);
        return StockMovementPersistenceMapper.toDomain(saved);
    }

    @Override
    public List<StockMovement> findByProductId(Long productId) {
        return jpaRepository.findByProductIdOrderByCreatedAtDesc(productId).stream()
                .map(StockMovementPersistenceMapper::toDomain)
                .collect(Collectors.toList());
    }

    @Override
    public List<StockMovement> findAll() {
        return jpaRepository.findAll().stream()
                .map(StockMovementPersistenceMapper::toDomain)
                .collect(Collectors.toList());
```

```
    }
}
```

A.6 Infraestructura: Seguridad y Configuración

backend/src/main/java/com/dongato/inventory/infrastructure/config/DataSeeder.java

Código fuente

```
package com.dongato.inventory.infrastructure.config;

import com.dongato.inventory.infrastructure.persistence.entity.CategoryEntity;
import com.dongato.inventory.infrastructure.persistence.entity.ProductEntity;
import com.dongato.inventory.infrastructure.persistence.repository.JpaCategoryRepository;
import com.dongato.inventory.infrastructure.persistence.repository.JpaProductRepository;
import com.dongato.inventory.domain.model.ProductStatus;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.math.BigDecimal;
import java.time.LocalDateTime;

/**
 * Seeds the database with initial cafeteria data on first run.
 * Only inserts if the database is empty.
 */
@Configuration
@RequiredArgsConstructor
@Slf4j
public class DataSeeder {

    @Bean
    public CommandLineRunner seedData(
            JpaCategoryRepository categoryRepo,
            JpaProductRepository productRepo) {
        return args -> {
            if (categoryRepo.count() > 0) {
                log.info("Database already seeded -- skipping");
                return;
            }

            log.info("Seeding initial data for Cafetería Don Gato...");
            LocalDateTime now = LocalDateTime.now();

            // Categories
            CategoryEntity cafe = categoryRepo.save(CategoryEntity.builder()
                    .name("CAFE").description("Bebidas de café preparadas").createdAt(now).build());
            CategoryEntity bebida = categoryRepo.save(CategoryEntity.builder()
                    .name("BEBIDA").description("Bebidas frías y calientes no-café").createdAt(now).build())
    ;
            CategoryEntity snack = categoryRepo.save(CategoryEntity.builder()
                    .name("SNACK").description("Bocadillos y acompañamientos").createdAt(now).build());

            // Products -- CAFE
            productRepo.save(ProductEntity.builder()
                    .name("Americano").price(new BigDecimal("2.50")).stock(50)
                    .status(ProductStatus.AVAILABLE).categoryId(cafe.getId()).createdAt(now).build());
            productRepo.save(ProductEntity.builder()
                    .name("Cappuccino").price(new BigDecimal("3.50")).stock(40)
                    .status(ProductStatus.AVAILABLE).categoryId(cafe.getId()).createdAt(now).build());
            productRepo.save(ProductEntity.builder()
                    .name("Latte").price(new BigDecimal("3.75")).stock(35)
                    .status(ProductStatus.AVAILABLE).categoryId(cafe.getId()).createdAt(now).build());
            productRepo.save(ProductEntity.builder()
                    .name("Espresso").price(new BigDecimal("2.00")).stock(60)
                    .status(ProductStatus.AVAILABLE).categoryId(cafe.getId()).createdAt(now).build());
```

```
            productRepo.save(ProductEntity.builder()
                    .name("Mocha").price(new BigDecimal("4.00")).stock(30)
                    .status(ProductStatus.AVAILABLE).categoryId(cafe.getId()).createdAt(now).build());

            // Products -- BEBIDA
            productRepo.save(ProductEntity.builder()
                    .name("Jugo de Naranja").price(new BigDecimal("2.50")).stock(25)
                    .status(ProductStatus.AVAILABLE).categoryId(bebida.getId()).createdAt(now).build());
            productRepo.save(ProductEntity.builder()
                    .name("Té Verde").price(new BigDecimal("2.00")).stock(40)
                    .status(ProductStatus.AVAILABLE).categoryId(bebida.getId()).createdAt(now).build());
            productRepo.save(ProductEntity.builder()
                    .name("Agua Mineral").price(new BigDecimal("1.50")).stock(100)
                    .status(ProductStatus.AVAILABLE).categoryId(bebida.getId()).createdAt(now).build());

            // Products -- SNACK
            productRepo.save(ProductEntity.builder()
                    .name("Croissant").price(new BigDecimal("2.75")).stock(20)
                    .status(ProductStatus.AVAILABLE).categoryId(snack.getId()).createdAt(now).build());
            productRepo.save(ProductEntity.builder()
                    .name("Muffin de Arándano").price(new BigDecimal("3.00")).stock(15)
                    .status(ProductStatus.AVAILABLE).categoryId(snack.getId()).createdAt(now).build());
            productRepo.save(ProductEntity.builder()
                    .name("Sándwich Club").price(new BigDecimal("5.50")).stock(10)
                    .status(ProductStatus.AVAILABLE).categoryId(snack.getId()).createdAt(now).build());

            log.info("Database seeded with {} categories and {} products",
                    categoryRepo.count(), productRepo.count());
        };
    }
}
```

backend/src/main/java/com/dongato/inventory/infrastructure/security/JwtAuthenticationFilter.java

Código fuente

```java
package com.dongato.inventory.infrastructure.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.extern.slf4j.Slf4j;
import org.springframework.context.annotation.Lazy;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.util.StringUtils;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

/**
 * JWT Authentication Filter -- intercepts every request to validate the Bearer token.
 * <p>
 * McCall Factor: Integrity -- enforces authentication on every request.
 */
@Component
@Slf4j
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtTokenProvider jwtTokenProvider;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(
            JwtTokenProvider jwtTokenProvider,
            @Lazy UserDetailsService userDetailsService) {
        this.jwtTokenProvider = jwtTokenProvider;
```

```
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain) throws ServletException, IOException {

        String token = extractTokenFromRequest(request);

        if (StringUtils.hasText(token) && jwtTokenProvider.validateToken(token)) {
            String username = jwtTokenProvider.getUsernameFromToken(token);
            UserDetails userDetails = userDetailsService.loadUserByUsername(username);

            UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(
                            userDetails, null, userDetails.getAuthorities());
            authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

            SecurityContextHolder.getContext().setAuthentication(authToken);
            log.debug("Authenticated user: {}", username);
        }

        filterChain.doFilter(request, response);
    }

    private String extractTokenFromRequest(HttpServletRequest request) {
        String bearerToken = request.getHeader("Authorization");
        if (StringUtils.hasText(bearerToken) && bearerToken.startsWith("Bearer ")) {
            return bearerToken.substring(7);
        }
        return null;
    }
}
```

backend/src/main/java/com/dongato/inventory/infrastructure/security/JwtTokenProvider.java

Código fuente

```
package com.dongato.inventory.infrastructure.security;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

/**
 * JWT utility for token generation and validation.
 * <p>
 * McCall Factor: Integrity -- secure authentication tokens.
 * Uses HMAC-SHA256 signing with configurable expiration.
 */
@Component
public class JwtTokenProvider {

    private final SecretKey secretKey;
    private final long expirationMs;

    public JwtTokenProvider(
            @Value("${jwt.secret:CafeteriaDonGatoSecretKeyForJWTToken2026MustBeAtLeast256Bits}") String
     secret,
            @Value("${jwt.expiration-ms:3600000}") long expirationMs) {
        this.secretKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.expirationMs = expirationMs;
    }

```

```
    /**
     * Generates a JWT token for the given username.
     */
    public String generateToken(String username) {
        Date now = new Date();
        Date expiry = new Date(now.getTime() + expirationMs);

        return Jwts.builder()
                .subject(username)
                .issuedAt(now)
                .expiration(expiry)
                .signWith(secretKey)
                .compact();
    }

    /**
     * Extracts the username from a JWT token.
     */
    public String getUsernameFromToken(String token) {
        return Jwts.parser()
                .verifyWith(secretKey)
                .build()
                .parseSignedClaims(token)
                .getPayload()
                .getSubject();
    }

    /**
     * Validates the JWT token.
     */
    public boolean validateToken(String token) {
        try {
            Jwts.parser()
                    .verifyWith(secretKey)
                    .build()
                    .parseSignedClaims(token);
            return true;
        } catch (JwtException | IllegalArgumentException e) {
            return false;
        }
    }

    public long getExpirationMs() {
        return expirationMs;
    }
}
```

backend/src/main/java/com/dongato/inventory/infrastructure/security/SecurityConfig.java

Código fuente

```
package com.dongato.inventory.infrastructure.security;

import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.
    AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
```

```java
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

/**
 * Spring Security configuration with JWT-based stateless authentication.
 * <p>
 * McCall Factor: Integrity -- enforces authentication and authorization.
 * Security hardening: CSRF disabled (stateless), CORS configured, session-less.
 */
@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
                .csrf(csrf -> csrf.disable()) // Stateless API -- CSRF not needed
                .cors(cors -> cors.configurationSource(corsConfigurationSource()))
                .sessionManagement(session ->
                        session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(auth -> auth
                        // Public endpoints
                        .requestMatchers("/api/v1/auth/**").permitAll()
                        .requestMatchers(HttpMethod.GET, "/api/v1/categories/**").permitAll()
                        .requestMatchers(HttpMethod.GET, "/api/v1/products/**").permitAll()
                        // All write operations require authentication
                        .anyRequest().authenticated()
                )
                .addFilterBefore(jwtAuthenticationFilter,
                        UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService(PasswordEncoder passwordEncoder) {
        // In-memory user for educational project -- production would use a DB-backed UserDetailsService
        var admin = User.builder()
                .username("admin")
                .password(passwordEncoder.encode("dongato2026"))
                .roles("ADMIN")
                .build();

        var user = User.builder()
                .username("cajero")
                .password(passwordEncoder.encode("cafe123"))
                .roles("USER")
                .build();

        return new InMemoryUserDetailsManager(admin, user);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(
            AuthenticationConfiguration authConfig) throws Exception {
        return authConfig.getAuthenticationManager();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(List.of("http://localhost:3000", "http://localhost:5173"));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        config.setAllowedHeaders(List.of("*"));
        config.setAllowCredentials(true);
```

```
        config.setMaxAge(3600L);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return source;
    }
}
```

A.7 Interfaces REST: Controladores

backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/AuthController.java

Código fuente

```java
package com.dongato.inventory.interfaces.rest.controller;

import com.dongato.inventory.infrastructure.security.JwtTokenProvider;
import com.dongato.inventory.interfaces.rest.dto.AuthRequestDTO;
import com.dongato.inventory.interfaces.rest.dto.AuthResponseDTO;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

/**
 * Authentication controller for JWT token generation.
 * <p>
 * McCall Factor: Integrity -- secure authentication endpoint.
 */
@RestController
@RequestMapping("/api/v1/auth")
@RequiredArgsConstructor
@Slf4j
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final JwtTokenProvider jwtTokenProvider;

    @PostMapping("/login")
    public ResponseEntity<AuthResponseDTO> login(@Valid @RequestBody AuthRequestDTO request) {
        try {
            Authentication authentication = authenticationManager.authenticate(
                    new UsernamePasswordAuthenticationToken(
                            request.getUsername(), request.getPassword()));

            String token = jwtTokenProvider.generateToken(authentication.getName());

            AuthResponseDTO response = AuthResponseDTO.builder()
                    .token(token)
                    .type("Bearer")
                    .username(authentication.getName())
                    .expiresIn(jwtTokenProvider.getExpirationMs() / 1000)
                    .build();

            log.info("User '{}' authenticated successfully", authentication.getName());
            return ResponseEntity.ok(response);

        } catch (BadCredentialsException e) {
            log.warn("Failed authentication attempt for user: {}", request.getUsername());
            throw e;
        }
```

```
    }
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/CategoryController.java

Código fuente

```java
package com.dongato.inventory.interfaces.rest.controller;

import com.dongato.inventory.application.usecases.CategoryUseCase;
import com.dongato.inventory.domain.model.Category;
import com.dongato.inventory.interfaces.rest.dto.CategoryCreateDTO;
import com.dongato.inventory.interfaces.rest.dto.CategoryResponseDTO;
import com.dongato.inventory.interfaces.rest.mapper.CategoryRestMapper;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.stream.Collectors;

/**
 * REST Controller for Category management.
 * <p>
 * McCall Factors: Interoperability (REST standard), Integrity (input validation).
 */
@RestController
@RequestMapping("/api/v1/categories")
@RequiredArgsConstructor
public class CategoryController {

    private final CategoryUseCase categoryUseCase;

    @PostMapping
    public ResponseEntity<CategoryResponseDTO> create(@Valid @RequestBody CategoryCreateDTO dto) {
        Category category = CategoryRestMapper.toDomain(dto);
        Category created = categoryUseCase.create(category);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(CategoryRestMapper.toResponseDto(created));
    }

    @GetMapping
    public ResponseEntity<List<CategoryResponseDTO>> findAll() {
        List<CategoryResponseDTO> categories = categoryUseCase.findAll().stream()
                .map(CategoryRestMapper::toResponseDto)
                .collect(Collectors.toList());
        return ResponseEntity.ok(categories);
    }

    @GetMapping("/{id}")
    public ResponseEntity<CategoryResponseDTO> findById(@PathVariable Long id) {
        Category category = categoryUseCase.findById(id);
        return ResponseEntity.ok(CategoryRestMapper.toResponseDto(category));
    }

    @PutMapping("/{id}")
    public ResponseEntity<CategoryResponseDTO> update(
            @PathVariable Long id,
            @Valid @RequestBody CategoryCreateDTO dto) {
        Category category = CategoryRestMapper.toDomain(dto);
        Category updated = categoryUseCase.update(id, category);
        return ResponseEntity.ok(CategoryRestMapper.toResponseDto(updated));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        categoryUseCase.delete(id);
        return ResponseEntity.noContent().build();
    }
```

```
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/ProductController.java

Código fuente

```java
package com.dongato.inventory.interfaces.rest.controller;

import com.dongato.inventory.application.usecases.ProductUseCase;
import com.dongato.inventory.domain.model.Product;
import com.dongato.inventory.domain.model.ProductStatus;
import com.dongato.inventory.interfaces.rest.dto.ProductCreateDTO;
import com.dongato.inventory.interfaces.rest.dto.ProductResponseDTO;
import com.dongato.inventory.interfaces.rest.dto.ProductUpdateDTO;
import com.dongato.inventory.interfaces.rest.mapper.ProductRestMapper;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.stream.Collectors;

/**
 * REST Controller for Product management.
 * <p>
 * McCall Factors: Interoperability (REST standard), Correctness (validated input).
 */
@RestController
@RequestMapping("/api/v1/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductUseCase productUseCase;

    @PostMapping
    public ResponseEntity<ProductResponseDTO> create(@Valid @RequestBody ProductCreateDTO dto) {
        Product product = ProductRestMapper.toDomain(dto);
        Product created = productUseCase.create(product);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(ProductRestMapper.toResponseDto(created));
    }

    @GetMapping
    public ResponseEntity<List<ProductResponseDTO>> findAll() {
        List<ProductResponseDTO> products = productUseCase.findAll().stream()
                .map(ProductRestMapper::toResponseDto)
                .collect(Collectors.toList());
        return ResponseEntity.ok(products);
    }

    @GetMapping("/{id}")
    public ResponseEntity<ProductResponseDTO> findById(@PathVariable Long id) {
        Product product = productUseCase.findById(id);
        return ResponseEntity.ok(ProductRestMapper.toResponseDto(product));
    }

    @GetMapping("/category/{categoryId}")
    public ResponseEntity<List<ProductResponseDTO>> findByCategory(@PathVariable Long categoryId) {
        List<ProductResponseDTO> products = productUseCase.findByCategoryId(categoryId).stream()
                .map(ProductRestMapper::toResponseDto)
                .collect(Collectors.toList());
        return ResponseEntity.ok(products);
    }

    @GetMapping("/status/{status}")
    public ResponseEntity<List<ProductResponseDTO>> findByStatus(@PathVariable ProductStatus status) {
        List<ProductResponseDTO> products = productUseCase.findByStatus(status).stream()
                .map(ProductRestMapper::toResponseDto)
                .collect(Collectors.toList());
```

```
        return ResponseEntity.ok(products);
    }

    @GetMapping("/search")
    public ResponseEntity<List<ProductResponseDTO>> searchByName(@RequestParam String name) {
        List<ProductResponseDTO> products = productUseCase.searchByName(name).stream()
                .map(ProductRestMapper::toResponseDto)
                .collect(Collectors.toList());
        return ResponseEntity.ok(products);
    }

    @PutMapping("/{id}")
    public ResponseEntity<ProductResponseDTO> update(
            @PathVariable Long id,
            @Valid @RequestBody ProductUpdateDTO dto) {
        Product product = ProductRestMapper.toDomain(dto);
        Product updated = productUseCase.update(id, product);
        return ResponseEntity.ok(ProductRestMapper.toResponseDto(updated));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        productUseCase.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/StockMovementController.java

Código fuente

```java
package com.dongato.inventory.interfaces.rest.controller;

import com.dongato.inventory.application.usecases.StockMovementUseCase;
import com.dongato.inventory.domain.model.StockMovement;
import com.dongato.inventory.interfaces.rest.dto.StockMovementCreateDTO;
import com.dongato.inventory.interfaces.rest.dto.StockMovementResponseDTO;
import com.dongato.inventory.interfaces.rest.mapper.StockMovementRestMapper;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.stream.Collectors;

/**
 * REST Controller for Stock Movement management.
 * <p>
 * This is the ONLY way to modify product stock -- ensuring full audit trail.
 * McCall Factors: Integrity (traceability), Interoperability (REST standard).
 */
@RestController
@RequestMapping("/api/v1/stock-movements")
@RequiredArgsConstructor
public class StockMovementController {

    private final StockMovementUseCase stockMovementUseCase;

    @PostMapping
    public ResponseEntity<StockMovementResponseDTO> register(
            @Valid @RequestBody StockMovementCreateDTO dto) {
        StockMovement movement = StockMovementRestMapper.toDomain(dto);
        StockMovement registered = stockMovementUseCase.registerMovement(movement);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(StockMovementRestMapper.toResponseDto(registered));
    }

    @GetMapping
    public ResponseEntity<List<StockMovementResponseDTO>> findAll() {
```

```
        List<StockMovementResponseDTO> movements = stockMovementUseCase.findAll().stream()
                .map(StockMovementRestMapper::toResponseDto)
                .collect(Collectors.toList());
        return ResponseEntity.ok(movements);
    }

    @GetMapping("/product/{productId}")
    public ResponseEntity<List<StockMovementResponseDTO>> findByProduct(
            @PathVariable Long productId) {
        List<StockMovementResponseDTO> movements = stockMovementUseCase.findByProductId(productId)
                .stream()
                .map(StockMovementRestMapper::toResponseDto)
                .collect(Collectors.toList());
        return ResponseEntity.ok(movements);
    }
}
```

A.8 Interfaces REST: DTOs, Errores y Mappers

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ApiErrorDTO.java

Código fuente

```java
package com.dongato.inventory.interfaces.rest.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;
import java.util.List;

/**
 * Standardized API error response.
 * McCall Factor: Usability -- consistent, informative error messages.
 */
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
@JsonInclude(JsonInclude.Include.NON_NULL)
public class ApiErrorDTO {

    private int status;
    private String error;
    private String message;
    private String path;
    private LocalDateTime timestamp;
    private List<FieldErrorDTO> fieldErrors;

    @Data
    @Builder
    @AllArgsConstructor
    @NoArgsConstructor
    public static class FieldErrorDTO {
        private String field;
        private String message;
    }
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/AuthRequestDTO.java

Código fuente

```java
package com.dongato.inventory.interfaces.rest.dto;

import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

/**
 * DTO for authentication requests.
 * McCall Factor: Integrity -- validates credentials at boundary.
 */
@Data
@AllArgsConstructor
@NoArgsConstructor
public class AuthRequestDTO {

    @NotBlank(message = "Username is required")
    private String username;

    @NotBlank(message = "Password is required")
    private String password;
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/AuthResponseDTO.java

Código fuente

```java
package com.dongato.inventory.interfaces.rest.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

/**
 * DTO for authentication responses (JWT token).
 * McCall Factor: Integrity -- secure token delivery.
 */
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class AuthResponseDTO {

    private String token;
    private String type;
    private String username;
    private long expiresIn;
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/CategoryCreateDTO.java

Código fuente

```java
package com.dongato.inventory.interfaces.rest.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
```

```
import lombok.NoArgsConstructor;

/**
 * DTO for creating a category.
 * McCall Factor: Correctness -- input validation at the boundary.
 */
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class CategoryCreateDTO {

    @NotBlank(message = "Category name is required")
    @Size(max = 100, message = "Category name must not exceed 100 characters")
    private String name;

    @Size(max = 255, message = "Description must not exceed 255 characters")
    private String description;
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/CategoryResponseDTO.java

Código fuente

```
package com.dongato.inventory.interfaces.rest.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/**
 * DTO for category responses.
 * McCall Factor: Interoperability -- standardized API contract.
 */
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class CategoryResponseDTO {

    private Long id;
    private String name;
    private String description;
    private LocalDateTime createdAt;
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ProductCreateDTO.java

Código fuente

```
package com.dongato.inventory.interfaces.rest.dto;

import jakarta.validation.constraints.*;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.math.BigDecimal;

/**
 * DTO for creating a product.
 * McCall Factor: Correctness -- strict input validation.
 */
```

```
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class ProductCreateDTO {

    @NotBlank(message = "Product name is required")
    @Size(max = 150, message = "Product name must not exceed 150 characters")
    private String name;

    @NotNull(message = "Price is required")
    @DecimalMin(value = "0.00", message = "Price must be non-negative")
    private BigDecimal price;

    @Min(value = 0, message = "Initial stock must be non-negative")
    private Integer stock;

    @NotNull(message = "Category ID is required")
    private Long categoryId;
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ProductResponseDTO.java

Código fuente

```
package com.dongato.inventory.interfaces.rest.dto;

import com.dongato.inventory.domain.model.ProductStatus;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.math.BigDecimal;
import java.time.LocalDateTime;

/**
 * DTO for product responses.
 * McCall Factor: Interoperability -- standardized API contract.
 */
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class ProductResponseDTO {

    private Long id;
    private String name;
    private BigDecimal price;
    private Integer stock;
    private ProductStatus status;
    private Long categoryId;
    private LocalDateTime createdAt;
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ProductUpdateDTO.java

Código fuente

```
package com.dongato.inventory.interfaces.rest.dto;

import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Builder;
```

```
import lombok.Data;
import lombok.NoArgsConstructor;

import java.math.BigDecimal;

/**
 * DTO for updating a product.
 * McCall Factor: Correctness -- validated partial updates.
 */
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class ProductUpdateDTO {

    @NotBlank(message = "Product name is required")
    @Size(max = 150, message = "Product name must not exceed 150 characters")
    private String name;

    @NotNull(message = "Price is required")
    @DecimalMin(value = "0.00", message = "Price must be non-negative")
    private BigDecimal price;

    private Long categoryId;
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/StockMovementCreateDTO.java

Código fuente

```
package com.dongato.inventory.interfaces.rest.dto;

import com.dongato.inventory.domain.model.MovementReason;
import com.dongato.inventory.domain.model.MovementType;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

/**
 * DTO for creating a stock movement.
 * McCall Factor: Integrity -- validates audit data at boundary.
 */
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class StockMovementCreateDTO {

    @NotNull(message = "Product ID is required")
    private Long productId;

    @NotNull(message = "Movement type is required (ENTRADA or SALIDA)")
    private MovementType type;

    @NotNull(message = "Quantity is required")
    @Min(value = 1, message = "Quantity must be at least 1")
    private Integer quantity;

    @NotNull(message = "Movement reason is required")
    private MovementReason reason;
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/StockMovementResponseDTO.java

Código fuente

```java
package com.dongato.inventory.interfaces.rest.dto;

import com.dongato.inventory.domain.model.MovementReason;
import com.dongato.inventory.domain.model.MovementType;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/**
 * DTO for stock movement responses.
 * McCall Factor: Interoperability -- consistent API response format.
 */
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class StockMovementResponseDTO {

    private Long id;
    private Long productId;
    private MovementType type;
    private Integer quantity;
    private MovementReason reason;
    private LocalDateTime createdAt;
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/exception/GlobalExceptionHandler.java

Código fuente

```java
package com.dongato.inventory.interfaces.rest.exception;

import com.dongato.inventory.domain.exception.BusinessRuleException;
import com.dongato.inventory.domain.exception.ResourceNotFoundException;
import com.dongato.inventory.interfaces.rest.dto.ApiErrorDTO;
import jakarta.servlet.http.HttpServletRequest;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.List;
import java.util.stream.Collectors;

/**
 * Global exception handler for the REST API.
 * <p>
 * McCall Factors:
 * - Usability: consistent, helpful error responses
 * - Integrity: no stack traces leaked to clients
 */
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiErrorDTO> handleNotFound(
            ResourceNotFoundException ex, HttpServletRequest request) {
```

```
        log.warn("Resource not found: {}", ex.getMessage());
        return buildResponse(HttpStatus.NOT_FOUND, ex.getMessage(), request);
    }

    @ExceptionHandler(BusinessRuleException.class)
    public ResponseEntity<ApiErrorDTO> handleBusinessRule(
            BusinessRuleException ex, HttpServletRequest request) {
        log.warn("Business rule violation: {}", ex.getMessage());
        return buildResponse(HttpStatus.CONFLICT, ex.getMessage(), request);
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<ApiErrorDTO> handleIllegalArgument(
            IllegalArgumentException ex, HttpServletRequest request) {
        log.warn("Invalid argument: {}", ex.getMessage());
        return buildResponse(HttpStatus.BAD_REQUEST, ex.getMessage(), request);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiErrorDTO> handleValidation(
            MethodArgumentNotValidException ex, HttpServletRequest request) {
        List<ApiErrorDTO.FieldErrorDTO> fieldErrors = ex.getBindingResult()
                .getFieldErrors().stream()
                .map(this::mapFieldError)
                .collect(Collectors.toList());

        ApiErrorDTO error = ApiErrorDTO.builder()
                .status(HttpStatus.BAD_REQUEST.value())
                .error("Validation Failed")
                .message("One or more fields have validation errors")
                .path(request.getRequestURI())
                .timestamp(LocalDateTime.now())
                .fieldErrors(fieldErrors)
                .build();

        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
    }

    @ExceptionHandler(BadCredentialsException.class)
    public ResponseEntity<ApiErrorDTO> handleBadCredentials(
            BadCredentialsException ex, HttpServletRequest request) {
        log.warn("Authentication failed: {}", ex.getMessage());
        return buildResponse(HttpStatus.UNAUTHORIZED, "Invalid username or password", request);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiErrorDTO> handleGeneric(
            Exception ex, HttpServletRequest request) {
        log.error("Unexpected error: {}", ex.getMessage(), ex);
        // Never expose internal details to client (Integrity)
        return buildResponse(HttpStatus.INTERNAL_SERVER_ERROR,
                "An unexpected error occurred", request);
    }

    private ResponseEntity<ApiErrorDTO> buildResponse(
            HttpStatus status, String message, HttpServletRequest request) {
        ApiErrorDTO error = ApiErrorDTO.builder()
                .status(status.value())
                .error(status.getReasonPhrase())
                .message(message)
                .path(request.getRequestURI())
                .timestamp(LocalDateTime.now())
                .build();
        return ResponseEntity.status(status).body(error);
    }

    private ApiErrorDTO.FieldErrorDTO mapFieldError(FieldError fieldError) {
        return ApiErrorDTO.FieldErrorDTO.builder()
                .field(fieldError.getField())
                .message(fieldError.getDefaultMessage())
                .build();
    }
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/mapper/CategoryRestMapper.java

Código fuente

```java
package com.dongato.inventory.interfaces.rest.mapper;

import com.dongato.inventory.domain.model.Category;
import com.dongato.inventory.interfaces.rest.dto.CategoryCreateDTO;
import com.dongato.inventory.interfaces.rest.dto.CategoryResponseDTO;

/**
 * Maps between Category domain model and REST DTOs.
 * McCall Factor: Maintainability -- isolates API contract from domain.
 */
public final class CategoryRestMapper {

    private CategoryRestMapper() {
        // Utility class
    }

    public static Category toDomain(CategoryCreateDTO dto) {
        return Category.builder()
                .name(dto.getName())
                .description(dto.getDescription())
                .build();
    }

    public static CategoryResponseDTO toResponseDto(Category domain) {
        return CategoryResponseDTO.builder()
                .id(domain.getId())
                .name(domain.getName())
                .description(domain.getDescription())
                .createdAt(domain.getCreatedAt())
                .build();
    }
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/mapper/ProductRestMapper.java

Código fuente

```java
package com.dongato.inventory.interfaces.rest.mapper;

import com.dongato.inventory.domain.model.Product;
import com.dongato.inventory.interfaces.rest.dto.ProductCreateDTO;
import com.dongato.inventory.interfaces.rest.dto.ProductResponseDTO;
import com.dongato.inventory.interfaces.rest.dto.ProductUpdateDTO;

/**
 * Maps between Product domain model and REST DTOs.
 * McCall Factor: Maintainability -- separates API layer from domain.
 */
public final class ProductRestMapper {

    private ProductRestMapper() {
        // Utility class
    }

    public static Product toDomain(ProductCreateDTO dto) {
        return Product.builder()
                .name(dto.getName())
                .price(dto.getPrice())
                .stock(dto.getStock() != null ? dto.getStock() : 0)
                .categoryId(dto.getCategoryId())
                .build();
    }

    public static Product toDomain(ProductUpdateDTO dto) {
        return Product.builder()
                .name(dto.getName())
```

```
                .price(dto.getPrice())
                .categoryId(dto.getCategoryId())
                .build();
    }

    public static ProductResponseDTO toResponseDto(Product domain) {
        return ProductResponseDTO.builder()
                .id(domain.getId())
                .name(domain.getName())
                .price(domain.getPrice())
                .stock(domain.getStock())
                .status(domain.getStatus())
                .categoryId(domain.getCategoryId())
                .createdAt(domain.getCreatedAt())
                .build();
    }
}
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/mapper/StockMovementRestMapper.java

Código fuente

```java
package com.dongato.inventory.interfaces.rest.mapper;

import com.dongato.inventory.domain.model.StockMovement;
import com.dongato.inventory.interfaces.rest.dto.StockMovementCreateDTO;
import com.dongato.inventory.interfaces.rest.dto.StockMovementResponseDTO;

/**
 * Maps between StockMovement domain model and REST DTOs.
 * McCall Factor: Maintainability -- isolates API contract from domain.
 */
public final class StockMovementRestMapper {

    private StockMovementRestMapper() {
        // Utility class
    }

    public static StockMovement toDomain(StockMovementCreateDTO dto) {
        return StockMovement.builder()
                .productId(dto.getProductId())
                .type(dto.getType())
                .quantity(dto.getQuantity())
                .reason(dto.getReason())
                .build();
    }

    public static StockMovementResponseDTO toResponseDto(StockMovement domain) {
        return StockMovementResponseDTO.builder()
                .id(domain.getId())
                .productId(domain.getProductId())
                .type(domain.getType())
                .quantity(domain.getQuantity())
                .reason(domain.getReason())
                .createdAt(domain.getCreatedAt())
                .build();
    }
}
```

A.9 SQA

backend/src/main/java/com/dongato/inventory/sqa/QualityScoringEngine.java

Código fuente

```java
package com.dongato.inventory.sqa;

/**
 * Quality Scoring Engine based on the McCall Quality Model.
 * <p>
 * Calculates a composite quality score from metrics collected during CI/CD.
 * Each factor is weighted according to project priorities.
 *
 * McCall Factors measured:
 * - Testability:      Code coverage percentage (JaCoCo)
 * - Correctness:      Bug count (SpotBugs)
 * - Maintainability:  Code smell count (SonarQube)
 * - Integrity:        Vulnerability count (OWASP Dependency Check)
 */
public class QualityScoringEngine {

    // Weights assigned based on project importance
    private static final double WEIGHT_CORRECTNESS     = 0.30;
    private static final double WEIGHT_TESTABILITY     = 0.30;
    private static final double WEIGHT_MAINTAINABILITY = 0.20;
    private static final double WEIGHT_INTEGRITY       = 0.20;

    private QualityScoringEngine() {
        // Utility class -- prevent instantiation
    }

    /**
     * Calculates the overall quality score (0-100).
     *
     * @param coveragePercent code coverage percentage from JaCoCo
     * @param bugs            bug count from SpotBugs
     * @param smells          code smell count from SonarQube
     * @param vulns           vulnerability count from OWASP Dependency Check
     * @return composite quality score (0-100)
     */
    public static double calculateScore(double coveragePercent, int bugs, int smells, int vulns) {
        if (coveragePercent < 0 || coveragePercent > 100) {
            throw new IllegalArgumentException("Coverage must be between 0 and 100");
        }
        if (bugs < 0 || smells < 0 || vulns < 0) {
            throw new IllegalArgumentException("Metric counts cannot be negative");
        }

        double testabilityScore      = Math.min(coveragePercent, 100.0);
        double correctnessScore      = Math.max(100.0 - (bugs * 10.0), 0.0);
        double maintainabilityScore  = Math.max(100.0 - (smells * 2.0), 0.0);
        double integrityScore        = vulns == 0 ? 100.0 : 0.0; // Zero tolerance

        return (testabilityScore     * WEIGHT_TESTABILITY)     +
               (correctnessScore     * WEIGHT_CORRECTNESS)     +
               (maintainabilityScore * WEIGHT_MAINTAINABILITY) +
               (integrityScore       * WEIGHT_INTEGRITY);
    }

    /**
     * Returns a human-readable quality grade.
     */
    public static String getGrade(double score) {
        if (score >= 90) return "A (Excellent)";
        if (score >= 80) return "B (Good)";
        if (score >= 70) return "C (Acceptable)";
        if (score >= 60) return "D (Needs Improvement)";
        return "F (Critical)";
    }
}
```

A.10 Pruebas

backend/src/test/java/com/dongato/inventory/DonGatoInventoryApplicationTests.java

Código fuente

```java
package com.dongato.inventory;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.test.context.TestPropertySource;

@SpringBootTest
@ActiveProfiles("test")
@TestPropertySource(properties = {
        "spring.docker.compose.enabled=false"
})
class DonGatoInventoryApplicationTests {

    @Test
    void contextLoads() {
        // Verifies that the Spring application context loads successfully
    }
}
```

backend/src/test/java/com/dongato/inventory/application/usecases/CategoryUseCaseTest.java

Código fuente

```java
package com.dongato.inventory.application.usecases;

import com.dongato.inventory.application.ports.out.CategoryRepositoryPort;
import com.dongato.inventory.domain.exception.BusinessRuleException;
import com.dongato.inventory.domain.exception.ResourceNotFoundException;
import com.dongato.inventory.domain.model.Category;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

/**
 * Unit tests for CategoryUseCase.
 * McCall Factor: Testability -- isolated business logic tests.
 */
@ExtendWith(MockitoExtension.class)
class CategoryUseCaseTest {

    @Mock
    private CategoryRepositoryPort categoryRepository;

    @InjectMocks
    private CategoryUseCase categoryUseCase;

    @Test
    @DisplayName("Should create category successfully")
    void shouldCreateCategorySuccessfully() {
        Category input = Category.builder().name("CAFE").description("Cafés").build();
        Category saved = Category.builder().id(1L).name("CAFE").description("Cafés").build();
```

```
39
40          when(categoryRepository.existsByName("CAFE")).thenReturn(false);
41          when(categoryRepository.save(any(Category.class))).thenReturn(saved);
42
43          Category result = categoryUseCase.create(input);
44
45          assertNotNull(result);
46          assertEquals(1L, result.getId());
47          assertEquals("CAFE", result.getName());
48          verify(categoryRepository).save(any(Category.class));
49      }
50
51      @Test
52      @DisplayName("Should throw when creating duplicate category")
53      void shouldThrowOnDuplicateCategory() {
54          Category input = Category.builder().name("CAFE").build();
55          when(categoryRepository.existsByName("CAFE")).thenReturn(true);
56
57          assertThrows(BusinessRuleException.class,
58                  () -> categoryUseCase.create(input));
59          verify(categoryRepository, never()).save(any());
60      }
61
62      @Test
63      @DisplayName("Should find category by ID")
64      void shouldFindById() {
65          Category expected = Category.builder().id(1L).name("CAFE").build();
66          when(categoryRepository.findById(1L)).thenReturn(Optional.of(expected));
67
68          Category result = categoryUseCase.findById(1L);
69
70          assertEquals("CAFE", result.getName());
71      }
72
73      @Test
74      @DisplayName("Should throw when category not found")
75      void shouldThrowWhenNotFound() {
76          when(categoryRepository.findById(99L)).thenReturn(Optional.empty());
77
78          assertThrows(ResourceNotFoundException.class,
79                  () -> categoryUseCase.findById(99L));
80      }
81
82      @Test
83      @DisplayName("Should return all categories")
84      void shouldFindAll() {
85          when(categoryRepository.findAll()).thenReturn(
86                  List.of(
87                          Category.builder().id(1L).name("CAFE").build(),
88                          Category.builder().id(2L).name("BEBIDA").build()
89                  ));
90
91          List<Category> result = categoryUseCase.findAll();
92          assertEquals(2, result.size());
93      }
94
95      @Test
96      @DisplayName("Should delete category successfully")
97      void shouldDeleteSuccessfully() {
98          when(categoryRepository.existsById(1L)).thenReturn(true);
99
100         assertDoesNotThrow(() -> categoryUseCase.delete(1L));
101         verify(categoryRepository).deleteById(1L);
102     }
103
104     @Test
105     @DisplayName("Should throw when deleting non-existent category")
106     void shouldThrowWhenDeletingNonExistent() {
107         when(categoryRepository.existsById(99L)).thenReturn(false);
108
109         assertThrows(ResourceNotFoundException.class,
110                 () -> categoryUseCase.delete(99L));
111     }
112 }
```

backend/src/test/java/com/dongato/inventory/application/usecases/StockMovementUseCaseTest.java

Código fuente

```java
package com.dongato.inventory.application.usecases;

import com.dongato.inventory.application.ports.out.ProductRepositoryPort;
import com.dongato.inventory.application.ports.out.StockMovementRepositoryPort;
import com.dongato.inventory.domain.exception.InsufficientStockException;
import com.dongato.inventory.domain.exception.ResourceNotFoundException;
import com.dongato.inventory.domain.model.*;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.math.BigDecimal;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

/**
 * Unit tests for StockMovementUseCase.
 * McCall Factor: Testability -- validates stock movement logic in isolation.
 */
@ExtendWith(MockitoExtension.class)
class StockMovementUseCaseTest {

    @Mock
    private StockMovementRepositoryPort movementRepository;

    @Mock
    private ProductRepositoryPort productRepository;

    @InjectMocks
    private StockMovementUseCase stockMovementUseCase;

    @Test
    @DisplayName("Should register ENTRADA and increase product stock")
    void shouldRegisterEntradaSuccessfully() {
        Product product = Product.builder()
                .id(1L).name("Americano").price(new BigDecimal("2.50"))
                .stock(10).status(ProductStatus.AVAILABLE).build();
        StockMovement movement = StockMovement.builder()
                .productId(1L).type(MovementType.ENTRADA)
                .quantity(5).reason(MovementReason.REPOSICION).build();
        StockMovement saved = StockMovement.builder()
                .id(1L).productId(1L).type(MovementType.ENTRADA)
                .quantity(5).reason(MovementReason.REPOSICION).build();

        when(productRepository.findById(1L)).thenReturn(Optional.of(product));
        when(productRepository.save(any(Product.class))).thenReturn(product);
        when(movementRepository.save(any(StockMovement.class))).thenReturn(saved);

        StockMovement result = stockMovementUseCase.registerMovement(movement);

        assertNotNull(result);
        assertEquals(15, product.getStock());
        assertEquals(ProductStatus.AVAILABLE, product.getStatus());
        verify(productRepository).save(product);
        verify(movementRepository).save(any(StockMovement.class));
    }

    @Test
    @DisplayName("Should register SALIDA and decrease product stock")
    void shouldRegisterSalidaSuccessfully() {
        Product product = Product.builder()
                .id(1L).name("Latte").price(new BigDecimal("3.75"))
                .stock(10).status(ProductStatus.AVAILABLE).build();
        StockMovement movement = StockMovement.builder()
                .productId(1L).type(MovementType.SALIDA)
```

```
                .quantity(3).reason(MovementReason.VENTA).build();
        StockMovement saved = StockMovement.builder()
                .id(2L).productId(1L).type(MovementType.SALIDA)
                .quantity(3).reason(MovementReason.VENTA).build();

        when(productRepository.findById(1L)).thenReturn(Optional.of(product));
        when(productRepository.save(any(Product.class))).thenReturn(product);
        when(movementRepository.save(any(StockMovement.class))).thenReturn(saved);

        StockMovement result = stockMovementUseCase.registerMovement(movement);

        assertNotNull(result);
        assertEquals(7, product.getStock());
        verify(productRepository).save(product);
    }

    @Test
    @DisplayName("Should throw InsufficientStockException on SALIDA with insufficient stock")
    void shouldThrowOnInsufficientStock() {
        Product product = Product.builder()
                .id(1L).name("Espresso").price(new BigDecimal("2.00"))
                .stock(2).status(ProductStatus.AVAILABLE).build();
        StockMovement movement = StockMovement.builder()
                .productId(1L).type(MovementType.SALIDA)
                .quantity(5).reason(MovementReason.VENTA).build();

        when(productRepository.findById(1L)).thenReturn(Optional.of(product));

        assertThrows(InsufficientStockException.class,
                () -> stockMovementUseCase.registerMovement(movement));
        verify(productRepository, never()).save(any());
        verify(movementRepository, never()).save(any());
    }

    @Test
    @DisplayName("Should throw when product not found")
    void shouldThrowWhenProductNotFound() {
        StockMovement movement = StockMovement.builder()
                .productId(99L).type(MovementType.ENTRADA)
                .quantity(5).reason(MovementReason.REPOSICION).build();

        when(productRepository.findById(99L)).thenReturn(Optional.empty());

        assertThrows(ResourceNotFoundException.class,
                () -> stockMovementUseCase.registerMovement(movement));
    }

    @Test
    @DisplayName("Should set status to OUT_OF_STOCK when stock reaches zero")
    void shouldSetOutOfStockWhenZero() {
        Product product = Product.builder()
                .id(1L).name("Mocha").price(new BigDecimal("4.00"))
                .stock(5).status(ProductStatus.AVAILABLE).build();
        StockMovement movement = StockMovement.builder()
                .productId(1L).type(MovementType.SALIDA)
                .quantity(5).reason(MovementReason.VENTA).build();
        StockMovement saved = StockMovement.builder()
                .id(3L).productId(1L).type(MovementType.SALIDA)
                .quantity(5).reason(MovementReason.VENTA).build();

        when(productRepository.findById(1L)).thenReturn(Optional.of(product));
        when(productRepository.save(any(Product.class))).thenReturn(product);
        when(movementRepository.save(any(StockMovement.class))).thenReturn(saved);

        stockMovementUseCase.registerMovement(movement);

        assertEquals(0, product.getStock());
        assertEquals(ProductStatus.OUT_OF_STOCK, product.getStatus());
    }
}
```

backend/src/test/java/com/dongato/inventory/domain/model/CategoryTest.java

Código fuente

```java
package com.dongato.inventory.domain.model;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

/**
 * Unit tests for Category domain model.
 * McCall Factor: Testability -- validates domain invariants.
 */
class CategoryTest {

    @Test
    @DisplayName("Should validate successfully with valid data")
    void shouldValidateSuccessfully() {
        Category category = Category.builder()
                .name("CAFE")
                .description("Bebidas de café")
                .build();

        assertDoesNotThrow(category::validate);
    }

    @Test
    @DisplayName("Should throw on null name")
    void shouldThrowOnNullName() {
        Category category = Category.builder()
                .name(null)
                .build();

        assertThrows(IllegalArgumentException.class, category::validate);
    }

    @Test
    @DisplayName("Should throw on blank name")
    void shouldThrowOnBlankName() {
        Category category = Category.builder()
                .name("   ")
                .build();

        assertThrows(IllegalArgumentException.class, category::validate);
    }
}
```

backend/src/test/java/com/dongato/inventory/domain/model/ProductTest.java

Código fuente

```java
package com.dongato.inventory.domain.model;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Nested;
import org.junit.jupiter.api.Test;

import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.*;

/**
 * Unit tests for Product domain model.
 * McCall Factor: Testability -- validates business rules in isolation.
 */
class ProductTest {

    @Nested
```

```
18      @DisplayName("Stock Management")
19      class StockManagement {
20
21          @Test
22          @DisplayName("Should add stock and recalculate status to AVAILABLE")
23          void shouldAddStockSuccessfully() {
24              Product product = Product.builder()
25                      .name("Americano")
26                      .price(new BigDecimal("2.50"))
27                      .stock(0)
28                      .status(ProductStatus.OUT_OF_STOCK)
29                      .build();
30
31              product.addStock(10);
32
33              assertEquals(10, product.getStock());
34              assertEquals(ProductStatus.AVAILABLE, product.getStatus());
35          }
36
37          @Test
38          @DisplayName("Should remove stock and recalculate status")
39          void shouldRemoveStockSuccessfully() {
40              Product product = Product.builder()
41                      .name("Cappuccino")
42                      .price(new BigDecimal("3.50"))
43                      .stock(5)
44                      .status(ProductStatus.AVAILABLE)
45                      .build();
46
47              product.removeStock(5);
48
49              assertEquals(0, product.getStock());
50              assertEquals(ProductStatus.OUT_OF_STOCK, product.getStatus());
51          }
52
53          @Test
54          @DisplayName("Should throw exception for insufficient stock")
55          void shouldThrowOnInsufficientStock() {
56              Product product = Product.builder()
57                      .name("Latte")
58                      .price(new BigDecimal("3.75"))
59                      .stock(3)
60                      .build();
61
62              IllegalStateException ex = assertThrows(
63                      IllegalStateException.class,
64                      () -> product.removeStock(5));
65
66              assertTrue(ex.getMessage().contains("Insufficient stock"));
67          }
68
69          @Test
70          @DisplayName("Should throw exception for negative entry quantity")
71          void shouldThrowOnNegativeEntryQuantity() {
72              Product product = Product.builder()
73                      .name("Espresso")
74                      .price(new BigDecimal("2.00"))
75                      .stock(10)
76                      .build();
77
78              assertThrows(IllegalArgumentException.class,
79                      () -> product.addStock(-5));
80          }
81
82          @Test
83          @DisplayName("Should throw exception for zero exit quantity")
84          void shouldThrowOnZeroExitQuantity() {
85              Product product = Product.builder()
86                      .name("Mocha")
87                      .price(new BigDecimal("4.00"))
88                      .stock(10)
89                      .build();
90
91              assertThrows(IllegalArgumentException.class,
92                      () -> product.removeStock(0));
```

```
        }
    }

    @Nested
    @DisplayName("Status Recalculation")
    class StatusRecalculation {

        @Test
        @DisplayName("Should set AVAILABLE when stock > 0")
        void shouldBeAvailableWithPositiveStock() {
            Product product = Product.builder().stock(1).build();
            product.recalculateStatus();
            assertEquals(ProductStatus.AVAILABLE, product.getStatus());
        }

        @Test
        @DisplayName("Should set OUT_OF_STOCK when stock = 0")
        void shouldBeOutOfStockWhenZero() {
            Product product = Product.builder().stock(0).build();
            product.recalculateStatus();
            assertEquals(ProductStatus.OUT_OF_STOCK, product.getStatus());
        }

        @Test
        @DisplayName("Should set OUT_OF_STOCK when stock is null")
        void shouldBeOutOfStockWhenNull() {
            Product product = Product.builder().stock(null).build();
            product.recalculateStatus();
            assertEquals(ProductStatus.OUT_OF_STOCK, product.getStatus());
        }
    }

    @Nested
    @DisplayName("Validation")
    class Validation {

        @Test
        @DisplayName("Should throw on blank name")
        void shouldThrowOnBlankName() {
            Product product = Product.builder()
                    .name("")
                    .price(new BigDecimal("2.50"))
                    .stock(0)
                    .build();

            assertThrows(IllegalArgumentException.class, product::validate);
        }

        @Test
        @DisplayName("Should throw on negative price")
        void shouldThrowOnNegativePrice() {
            Product product = Product.builder()
                    .name("Test")
                    .price(new BigDecimal("-1.00"))
                    .stock(0)
                    .build();

            assertThrows(IllegalArgumentException.class, product::validate);
        }

        @Test
        @DisplayName("Should throw on negative stock")
        void shouldThrowOnNegativeStock() {
            Product product = Product.builder()
                    .name("Test")
                    .price(new BigDecimal("2.50"))
                    .stock(-5)
                    .build();

            assertThrows(IllegalArgumentException.class, product::validate);
        }

        @Test
        @DisplayName("Should pass validation for valid product")
        void shouldPassValidation() {
```

```
            Product product = Product.builder()
                    .name("Americano")
                    .price(new BigDecimal("2.50"))
                    .stock(10)
                    .build();

            assertDoesNotThrow(product::validate);
        }
    }
}
```

backend/src/test/java/com/dongato/inventory/domain/model/StockMovementTest.java

Código fuente

```
package com.dongato.inventory.domain.model;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

/**
 * Unit tests for StockMovement domain model.
 * McCall Factor: Testability -- validates audit data integrity.
 */
class StockMovementTest {

    @Test
    @DisplayName("Should validate successfully with valid data")
    void shouldValidateSuccessfully() {
        StockMovement movement = StockMovement.builder()
                .productId(1L)
                .type(MovementType.ENTRADA)
                .quantity(10)
                .reason(MovementReason.REPOSICION)
                .build();

        assertDoesNotThrow(movement::validate);
    }

    @Test
    @DisplayName("Should throw on null product ID")
    void shouldThrowOnNullProductId() {
        StockMovement movement = StockMovement.builder()
                .productId(null)
                .type(MovementType.ENTRADA)
                .quantity(10)
                .reason(MovementReason.REPOSICION)
                .build();

        assertThrows(IllegalArgumentException.class, movement::validate);
    }

    @Test
    @DisplayName("Should throw on null type")
    void shouldThrowOnNullType() {
        StockMovement movement = StockMovement.builder()
                .productId(1L)
                .type(null)
                .quantity(10)
                .reason(MovementReason.VENTA)
                .build();

        assertThrows(IllegalArgumentException.class, movement::validate);
    }

    @Test
    @DisplayName("Should throw on zero quantity")
    void shouldThrowOnZeroQuantity() {
        StockMovement movement = StockMovement.builder()
```

```
                .productId(1L)
                .type(MovementType.SALIDA)
                .quantity(0)
                .reason(MovementReason.VENTA)
                .build();

        assertThrows(IllegalArgumentException.class, movement::validate);
    }

    @Test
    @DisplayName("Should throw on null reason")
    void shouldThrowOnNullReason() {
        StockMovement movement = StockMovement.builder()
                .productId(1L)
                .type(MovementType.ENTRADA)
                .quantity(10)
                .reason(null)
                .build();

        assertThrows(IllegalArgumentException.class, movement::validate);
    }
}
```

backend/src/test/java/com/dongato/inventory/sqa/QualityScoringEngineTest.java

Código fuente

```
package com.dongato.inventory.sqa;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

/**
 * Unit tests for QualityScoringEngine.
 * McCall Factor: Testability -- validates quality scoring calculations.
 */
class QualityScoringEngineTest {

    @Test
    @DisplayName("Should return perfect score with ideal metrics")
    void shouldReturnPerfectScore() {
        double score = QualityScoringEngine.calculateScore(100.0, 0, 0, 0);
        assertEquals(100.0, score, 0.01);
        assertEquals("A (Excellent)", QualityScoringEngine.getGrade(score));
    }

    @Test
    @DisplayName("Should return zero integrity score with vulnerabilities")
    void shouldPenalizeVulnerabilities() {
        double withVulns = QualityScoringEngine.calculateScore(100.0, 0, 0, 1);
        double withoutVulns = QualityScoringEngine.calculateScore(100.0, 0, 0, 0);

        assertTrue(withVulns < withoutVulns);
        assertEquals(80.0, withVulns, 0.01); // 100*0.3 + 100*0.3 + 100*0.2 + 0*0.2
    }

    @Test
    @DisplayName("Should degrade correctness score with bugs")
    void shouldDegradeWithBugs() {
        double score = QualityScoringEngine.calculateScore(100.0, 5, 0, 0);
        // Correctness: max(100 - 50, 0) = 50
        // 100*0.3 + 50*0.3 + 100*0.2 + 100*0.2 = 30+15+20+20 = 85
        assertEquals(85.0, score, 0.01);
    }

    @Test
    @DisplayName("Should degrade maintainability with code smells")
    void shouldDegradeWithSmells() {
        double score = QualityScoringEngine.calculateScore(100.0, 0, 25, 0);
```

```
        // Maintainability: max(100 - 50, 0) = 50
        // 100*0.3 + 100*0.3 + 50*0.2 + 100*0.2 = 30+30+10+20 = 90
        assertEquals(90.0, score, 0.01);
    }

    @Test
    @DisplayName("Should throw on invalid coverage")
    void shouldThrowOnInvalidCoverage() {
        assertThrows(IllegalArgumentException.class,
                () -> QualityScoringEngine.calculateScore(-1.0, 0, 0, 0));
        assertThrows(IllegalArgumentException.class,
                () -> QualityScoringEngine.calculateScore(101.0, 0, 0, 0));
    }

    @Test
    @DisplayName("Should throw on negative metrics")
    void shouldThrowOnNegativeMetrics() {
        assertThrows(IllegalArgumentException.class,
                () -> QualityScoringEngine.calculateScore(80.0, -1, 0, 0));
    }

    @Test
    @DisplayName("Should grade correctly at boundaries")
    void shouldGradeCorrectly() {
        assertEquals("A (Excellent)", QualityScoringEngine.getGrade(90));
        assertEquals("B (Good)", QualityScoringEngine.getGrade(85));
        assertEquals("C (Acceptable)", QualityScoringEngine.getGrade(75));
        assertEquals("D (Needs Improvement)", QualityScoringEngine.getGrade(65));
        assertEquals("F (Critical)", QualityScoringEngine.getGrade(50));
    }
}
```

Apéndice B

Snapshots de Archivos Eliminados

B.1 Snapshot de aplicación Spring anterior

backend/src/main/java/com/sqa_inventory/backend/BackendApplication.java

Archivo eliminado en el commit

```java
package com.sqa_inventory.backend;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BackendApplication {

	public static void main(String[] args) {
		SpringApplication.run(BackendApplication.class, args);
	}

}
```

B.2 Snapshot de prueba anterior

backend/src/test/java/com/sqa_inventory/backend/BackendApplicationTests.java

Archivo de prueba eliminado en el commit

```java
package com.sqa_inventory.backend;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class BackendApplicationTests {

	@Test
	void contextLoads() {
	}

}
```